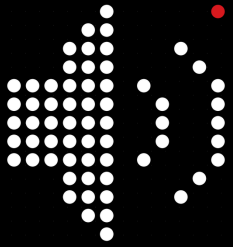
CURRENTLY IN EXPERIMENTAL BETA AND IN TESTING MODE



CMF TRUE STEREO

A MAGISK MODULE FOR REAL LEFT AND RIGHT STEREO ON THE CMF PHONE 1

THE EARPIECE AND THE BOTTOM SPEAKER PLAY AS A STEREO PAIR
LEFT AND RIGHT FOLLOW THE SCREEN, HOWEVER YOU TURN THE PHONE
MONO MODE: BOTH SPEAKERS PLAY THE FULL MIX, LOUDER AND FULLER
SPEAKER AND EARPIECE PROTECTION · UNDOES ITSELF AFTER TWO FAILED BOOTS
NO CUSTOM KERNEL, JUST ROOT · MAGISK, KERNELSU AND APATCH

©DR.ESSENCE · NDOT FONTS ARE OWNED BY ©NOTHING

INSTRUCTION MANUAL

V1.0.9-BETA · MANUAL IN EARLY BETA

REAL LEFT AND RIGHT STEREO FOR THE CMF PHONE 1

CONTENTS

01 WELCOME 10

1.1 What CMF True Stereo is 11

1.2 The two modes at a glance 11

1.3 What you need 12

1.4 The phone and build it was tested on 13

1.5 Safety first 14

1.6 What it does 15

1.7 What it does not do 15

1.8 The companion app, CMF Stereo 16

1.9 This version and its beta status 17

1.10 How to read this manual 17

02 INSTALLING 19

2.1 Install in three steps 20

2.2 Getting the zip and checking it 20

2.3 Installing with Magisk 21

2.4 Installing with KernelSU or APatch 21

2.5 What the installer does and prints 22

2.6 The first boot 24

2.7 Checking that it works 25

2.8 Installing the app 26

2.9 Installing on another phone 27

2.10 Updating the module 28

2.11 Updating from older versions 29
2.12 Uninstalling 29

03 EVERYDAY USE 31

3.1 Where the controls are 32
3.2 The module's card 32
3.3 The Action button 33
3.4 Turning the module on and off 34
3.5 While playing, or always on 35
3.6 Headphones and Bluetooth 35
3.7 Phone calls 36
3.8 Turning the phone: left and right follow the screen 37
3.9 Testing with the test files 37
3.10 Apps that were checked 38
3.11 Nothing Settings and the apps stay in step 39
3.12 Battery and background work 39

04 MONO AND STEREO 41

4.1 Mono 42
4.2 Stereo 42
4.3 Which mode to choose 43
4.4 Switching between Mono and Stereo 43
4.5 What you hear during a switch 44
4.6 When a switch waits for a reboot 44
4.7 What a live switch checks 45
4.8 What Stereo changes, and nothing else 46

4.9 The first second of a stream 48
4.10 No vendor file ships in this module 48
4.11 Stereo after a Nothing OS update 49
4.12 Known limits in Stereo 50

05 PROTECTION AND LEVELS 51

5.1 Why Stereo needs its own protection 52
5.2 Risks, plainly 53
5.3 The speaker level and the 6 dB margin 54
5.4 Speaker boost 55
5.5 Deep bass guard 56
5.6 Earpiece limiter 57
5.7 Earpiece leveller 58
5.8 When the protection runs 59
5.9 The earpiece level 60
5.10 Balancing the earpiece and the speaker 61
5.11 Awinic compared with the module 62
5.12 Choosing your settings 63

06 THE COMMAND LINE: STEREOCTL 64

6.1 Opening a root shell 65
6.2 All commands at a glance 65
6.3 status 67
6.4 status brief 68
6.5 doctor 69
6.6 channels 70

6.7 protect 71
6.8 protect boost 72
6.9 protect bass 72
6.10 protect earpiece 73
6.11 protect leveller 73
6.12 on and off 73
6.13 solo and unsolo 74
6.14 guard 75
6.15 apply and revert 76
6.16 restart 76
6.17 reset 77
6.18 log 77
6.19 edit 78
6.20 probe 79
6.21 report 80
6.22 scan 80
6.23 ctl 81
6.24 dump 82
6.25 diff 82
6.26 xml-patch and xml-revert 83

07 CONFIGURATION FILES 85

7.1 Where the files live 86
7.2 Editing the settings safely 86
7.3 All settings at a glance 87
7.4 ENABLED 88

7.5 MODE 89
7.6 WATCH_INTERVAL 89
7.7 STARTUP_DELAY 90
7.8 REQUIRE_SPEAKER_ROUTE 90
7.9 GUARD_CTL and GUARD_VALUE 91
7.10 PLAYBACK_CTL and PLAYBACK_VALUE 91
7.11 MIXER_CARD 92
7.12 CHANNELS 93
7.13 STEREO_ATTEN_DB 94
7.14 The protection settings 94
7.15 LOG_LEVEL and LOG_MAX_KB 95
7.16 actions.conf: the routing 95
7.17 The CMF Phone 1 routing profile 96

08 SAFETY AND RECOVERY 98

8.1 The safety nets at a glance 99
8.2 Boot watchdog 100
8.3 Crash-loop guard 100
8.4 Hash checks on every file 101
8.5 The unknown-build guard 102
8.6 Ships empty on other devices 102
8.7 XML validation 103
8.8 Everything put back 103
8.9 Manual recovery 104
8.10 Reporting a security problem 105

09 TROUBLESHOOTING 106

9.1 First steps for any problem 107
9.2 The earpiece is silent 107
9.3 Reading doctor's messages 108
9.4 No stereo: both drivers play the same 109
9.5 Stereo refuses to start 109
9.6 Left and right are reversed 110
9.7 Distortion, crackle or rattle 111
9.8 The sound stops, drops out or pauses 112
9.9 Full volume not reached: check other modules 112
9.10 The earpiece is weak or uneven 113
9.11 The card says Not running, or the daemon misbehaves 113
9.12 Back in Mono by itself 114
9.13 tinymix problems 115
9.14 What to send when asking for help 115

10 QUESTIONS AND ANSWERS 116

10.1 Compatibility 117
10.2 Sound 117
10.3 Safety 118
10.4 Everyday questions 119

11 HOW IT WORKS 120

11.1 In plain words 121
11.2 Three layers, smallest hammer first 121
11.3 How the earpiece was brought in 122
11.4 The two places the phone folds to mono 123

11.5 From the app to the two drivers 124
11.6 The protection library 124
11.7 The background daemon 125
11.8 How turning the phone is handled 126
11.9 How playback is detected 127
11.10 How a live switch works 128
11.11 What the boot and button scripts do 129

12 TECHNICAL REFERENCE 131

12.1 Findings: where the speaker is folded to mono 132
12.2 Findings: protection and the drivers 133
12.3 Mixer controls and hardware 134
12.4 System properties used 135
12.5 The HAL patch 135
12.6 Files Stereo replaces, and their hashes 136
12.7 tinymix 137
12.8 The probe's output files 138
12.9 The module's files 139
12.10 The data folder's files 140
12.11 MediaTek's second-loudspeaker machinery 141
12.12 Development checks 142
12.13 Test status of this version 143
12.14 Version history 144

13 CREDITS AND LICENCE 146

13.1 Author 147

13.2 The Hi-Res Audio module 147
13.3 A bug reported upstream 148
13.4 Why its stereo approach does not work here 149
13.5 Others who did this before 149
13.6 Licence 149



WELCOME

What CMF True Stereo is, what it needs, and the few things to know before you play anything loud.

1.1 What CMF True Stereo is

CMF True Stereo is a root module for the CMF Phone 1. It makes the phone play media from its earpiece as well as its bottom speaker.

In Stereo the two become a real left and right pair. Left and right follow the screen, however you turn the phone. Games, films and music finally have sides.

You do not need a custom kernel. You only need root, through Magisk, KernelSU or APatch.

- It never writes to the vendor partition.
- It only patches files it has checked are the exact ones it was built for.
- It undoes its stereo change by itself after two failed boots.

> TECHNICAL
>
> **Names.** The project is called CMF-true-stereo (it was once called cmf-fake-stereo, when it only did what Mono does today). In your root manager the module is listed as CMF Phone 1 Dual Speaker (mono / stereo), by Dr.Essence, with the module id cmf_stereo. The command-line tool is stereoctl.

1.2 The two modes at a glance

Mode	What you hear	Speaker protection

Mono (default)	Both the earpiece and the speaker play the full mix. Louder, fuller and taller than the bottom speaker alone.	Awinic's own speaker protection, as on a stock phone.
Stereo	The earpiece and the speaker play left and right, following how you hold the phone.	Awinic's protection is off for media (calls keep it). The module's own protection stands in.

You can switch between them at any time, and the change happens at once, without a reboot. Chapter 4 explains both modes in full.

1.3 What you need

- A **CMF Phone 1** (A015).
- **Root** through Magisk, KernelSU or APatch.
- For the **Action button** on the module's card: Magisk 28 or newer, KernelSU or APatch.
- Optional: the companion app **CMF Stereo** (or **CMF Stereo Pro**), if you would rather not use a terminal.
- Optional: a terminal with root, such as Termux with `su`, or `adb shell` then `su`.

> TECHNICAL
>
> The installer expects Android 12 or newer (API 31). On older Android it prints `! Android 12 or newer expected; continuing anyway.` and carries on.

1.4 The phone and build it was tested on

Everything in this manual was measured on one CMF Phone 1 (A015), EEA region:

Part	Version
Nothing OS	4.1, Tetris-B4.1-260615-1652-EEA (ro.build.display.id B4.1-260615-1652, product TetrisEEA)
Android	16 (BP2A.250605.031.A3), security patch 2026-06-01
Vendor	UP1A.231005.007 (the Android 14 vendor image, where the audio HAL is)
Chip	MediaTek MT6878 (code name Tetris)

Other regions (Global, India) are untested. The module checks the audio files by hash and only patches the exact ones it was built for. On a build whose files differ, it refuses Stereo and stays in Mono.

1.5 Safety first

WARNING

Read this before you use Stereo.

- **Stereo turns off Awinic's speaker protection for media.** Calls always keep it. In its place the module holds the speaker 6 dB below the amp's full output at its loudest (app speaker level 10), and 2 dB more for each level down. Its deepest bass is held back when it gets loud, and the earpiece is kept from clipping.
- **These are fixed limits.** There is no temperature model of the speaker in Stereo, as Awinic has in Mono.
- **Use Mono for long, loud listening.** In Stereo, lower the speaker level and back off at the first distortion.
- **The earpiece is a receiver.** It is a small driver with a fraction of the speaker's movement and power handling, and it has no protection circuit of its own. In Stereo the module's earpiece limiter (on by default) keeps it below where it crackles. In Mono the earpiece plays without that limiter.
- **Distortion or rattle means back off.** That kind of damage does not come back.

If the phone fails to boot twice in a row with Stereo on, the module removes Stereo by itself and falls back to Mono. Chapter 8 lists every safety net.

1.6 What it does

- Real stereo in Stereo mode: the earpiece and the bottom speaker play left and right, following the screen.
- Mono mode: both speakers play the full mix, with Awinic's speaker protection on the speaker.
- Plays media through the earpiece alongside the speaker, automatically.
- Drives the earpiece at the receiver amp's maximum, +8 dB, the same value the audio HAL itself uses.
- Works only while audio is playing (by default), so the earpiece is not held energised all day.
- Steps aside for headphones and Bluetooth.
- Puts back every control it touches when it stops, is turned off, or is uninstalled.
- Lets you switch modes with `stereoctl channels mono|stereo`, the app, or the Action button on the module's card.
- Shows on the module's card what is running: `[Stereo]`, `[Mono]`, `[Off]`, or `[Stereo unavailable]` after a Nothing OS update.

1.7 What it does not do

- **No Awinic protection on media in Stereo.** Awinic's processing on the audio DSP is the part that folds the sound to mono, so Stereo replaces it with the module's own, simpler protection. Awinic's live model of the speaker, built from the amp's current and voltage sense, is not copied.

- **No extra volume from the main speaker.** The speaker's own path is untouched; in Stereo it is in fact held a little lower.
- **Nothing on other phones out of the box.** On any other device it installs inert, and ships a probe to work out that device's routing.
- **No custom kernel**, and no write to the vendor partition.
- **No vendor file is shipped.** Stereo is built from your own phone's files, after checking them.

1.8 The companion app, CMF Stereo

The module works fully on its own. The app is an optional, friendly front end. It comes in two editions that can be installed side by side:

	CMF Stereo	CMF Stereo Pro
For	everyday listening	tuning and diagnosing
Controls	on/off, Mono/Stereo, earpiece and speaker level 1–10, speaker boost, deep bass guard, earpiece leveller and limiter	the same in raw values, plus solo, doctor, mode, mixer tools, diff, probe, logs and the config files
Look	Nothing style, NDot type in green	Material dark

Both offer a Quick Settings tile. With LSPosed, each can also add a **Stereo speakers** row in Nothing's Settings › Sound and a full Stereo page inside Settings.

Everything the apps do is a call to the module's own `stereoctl`, or an edit to the files in `/data/adb/cmf-stereo/`. The module stays the source of truth, and the terminal keeps working exactly as before.

> TIP
> The app needs root, superuser permission for itself, and the module flashed and rebooted first. Its own manual is in its repository, CMF-true-stereo-manager.

1.9 This version and its beta status

This manual describes **v1.0.9-beta** (internal version 10900). Versions are marked beta from this release on, because the module is still in testing. Nothing in the module changed since v1.0.8; the new number lets your root manager offer it as an update.

Stereo works and is in daily use on the test phone. Some features have been tested only on fake files, not on a phone yet; section 12.13 lists them.

1.10 How to read this manual

Each section starts in plain language. Precise technical detail follows in boxes like these:

> TIP
> **Tip.** Something useful, not essential.

> WARNING
>
> **Safety warning.** Read these. They protect your speaker, your earpiece and your ears.

> TECHNICAL
>
> **Technical detail.** Exact file names, values and mechanisms, for those who want them.

Commands are shown like `stereoctl status`. Run them in a root shell (see section 6.1). Example output marked `<pid>` or similar stands for a value that differs on your phone.



INSTALLING

Flash, reboot, play something. This chapter walks through each step, shows what the installer tells you, and covers updates, other phones and removal.

2.1 Install in three steps

1. Flash the zip in Magisk, KernelSU or APatch, and reboot.
2. Choose Stereo if you want it: open CMF Stereo and pick Stereo, or tap Action on the module's card, or run `su -c stereoctl channels stereo` in a terminal. It switches at once: the sound stops for a moment, then press play again.
3. Turn the phone sideways either way. Left and right follow the screen.

On a CMF Phone 1 that is the whole setup: the verified routing is preconfigured. After the first reboot the phone is in Mono, the default.

2.2 Getting the zip and checking it

Download the module from the project's releases page (CMF-true-stereo, latest release). If you build it yourself, `./build.sh` in the repository makes the flashable zip.

Every release lists the SHA-256 checksum of each file, in the release description and in `SHA256SUMS.txt`. Compare it with your download to be sure it is the genuine file.

```
sha256sum CMF-true-stereo-v1.0.9-beta.zip
```

> TIP
>
> Every release also carries two test files, `LEFT_ONLY_3min.wav` and `LEFT10s_RIGHT10s.wav`. Keep them: section 3.9 shows how to use them.

> TECHNICAL
>
> The exact zip file name on the release page may differ from the example above; use the name of the file you downloaded.

2.3 Installing with Magisk

1. Open Magisk and go to its **Modules** page.
2. Install the module zip from storage.
3. Read the installer's output (section 2.5), then reboot.

After the reboot the module's card shows its state in its first words, for example `[Mono] Both speakers play the full mix.` The **Action** button on the card needs Magisk 28 or newer.

From v0.9.6 on, Magisk shows an **Update** button on the card when a new release is out, and installs it straight from the release page.

2.4 Installing with KernelSU or APatch

Flash the same zip from the Modules page of KernelSU or APatch, then reboot. Everything in this manual works the same way:

- the module's card shows the live state;
- the **Action** button switches between Mono and Stereo;
- updates are offered on the card;

- the disable file for manual recovery is honoured (section 8.9).

2.5 What the installer does and prints

The installer deliberately does nothing to the audio stack. It lays down the tools and the settings, and tells you what to do next. On a CMF Phone 1 you will see lines like these:

```
CMF Phone 1 - stereo speaker
--------------------------
device   : <model> (<device>)
platform : <platform> / <manufacturer>
android  : <version> (API <n>)

installed default stereo.conf
guard: earpiece follows the speaker amp
playback detected on every DSP playback task (music, fast, primary, ...)
installed the CMF Phone 1 routing profile
channels : mono (stock). 'stereoctl channels stereo' for L/R

tinymix  : bundled (this device ships none)
           static aarch64 build of upstream tinyalsa;
           provenance in bin/README.md
actions  : 3 configured

Next steps
----------
1. reboot
2. play something
3. stereoctl status   (should show the routing applied)

Earpiece gain: Handset Volume 0 = +8 dB (loudest),
18 = -10 dB. Raise the number in actions.conf for a
quieter earpiece; back off at the first buzz.
```

TECHNICAL

Step by step, the installer:

1. checks the device: it looks for `Tetris`, `A015` or `CMF` in `ro.product.device` and `ro.product.model`;
2. creates `/data/adb/cmf-stereo/` with `state/` and `bin/`, and installs `stereo.conf` there (or keeps yours, section 2.11);
3. on a CMF Phone 1, sets the guard (`GUARD_CTL='aw_dev_0_switch'`, `GUARD_VALUE='Enable'`) and the playback detector (the six DSP task switches, section 7.10);
4. installs the CMF Phone 1 routing profile as `actions.conf`, unless you already have one with real content;
5. resets the boot watchdog counter (`boot_pending` = 0) and sets file permissions;
6. if your settings say `CHANNELS=stereo`, stages Stereo again from your phone's files (it refuses, and sets Mono, if they do not check out);
7. reports which `tinymix` will be used and how many actions are configured.

2.6 The first boot

After the reboot the module waits for Android to finish starting, then a further 10 seconds (`STARTUP_DELAY`), before it touches anything. This lets the audio system settle.

From then on a small background service (the daemon) watches for sound. When something plays on the speaker, it switches the earpiece on; when everything stops,

it switches it off again.

> TECHNICAL
>
> At boot, `post-fs-data.sh` runs first (before Magisk mounts anything): the boot watchdog and the unknown-build guard. Then `service.sh` waits for `sys.boot_completed` (up to 180 s), marks the boot healthy, keeps a hash-checked copy of the stock audio files, notes whether this is the analysed build, updates the card's text, and starts `scripts/stereo-daemon.sh`. Section 11.11 has the details.

2.7 Checking that it works

Play some music, then open a root shell and run:

```
su
stereoctl status    # all three controls should show want= matching now=
stereoctl solo      # speaker off, earpiece only - proves it is really the earpiece
stereoctl unsolo    # back to normal
```

In `status`, look at the `current values:` list at the end. Each of the three routing controls should show the same value after `want=` and `now=`.

`solo` turns the speaker off so you can hear the earpiece on its own. Always finish with `unsolo`.

> TIP
>
> If the earpiece stays silent, run `stereoctl doctor` with music playing. It walks every layer that can cause silence and names the one responsible (section 9.2).

2.8 Installing the app

Install **CMF Stereo** or **CMF Stereo Pro** (or both) from the app's releases page, CMF-true-stereo-manager, after the module is flashed and the phone rebooted. Grant it superuser permission when your root manager asks.

- The app can also be kept up to date through Komi Store, an app store for apps released on GitHub.
- For the Stereo speakers page inside Nothing Settings you need LSPosed, with the app on internal storage. Enable one edition in LSPosed (scope: Settings), not both, then force-stop Settings.
- The Quick Settings tile needs no LSPosed.

> TIP
>
> App versions matter for some settings: the protection settings need app v1.0.56 or newer (module v1.0.4+), the earpiece leveller app v1.0.60 or newer (module v1.0.5+), and the Settings page that follows every change at once app v1.0.65 or newer (module v1.0.6+).

2.9 Installing on another phone

On a phone that is not a CMF Phone 1 the module installs and changes nothing. The installer says so:

```
  ! This does not look like a CMF Phone 1.
    Nothing is applied automatically, so it is safe to keep
    going - the probe works on any device.
...
  installed empty actions.conf (run 'stereoctl probe')
...
  Until actions.conf has routing in it, this module changes
  nothing about how your phone sounds.
```

What you can do there is probe the audio hardware and share the report, so the routing for that device can be worked out:

```
su
stereoctl probe
stereoctl report
```

Section 8.6 explains why it ships empty, and sections 6.20 and 6.21 explain the two commands.

WARNING

Do not fill `actions.conf` with guessed control names. A wrong mixer write is, from least to most regret: nothing, silence, a crash of the audio HAL, or an earpiece driven past what a receiver is built for. When you find a working route, start its gain low and raise it one step at a time.

2.10 Updating the module

When a new release is out, your root manager shows an **Update** button on the module's card. Tap it, let it install, and reboot. You can also flash the new zip by hand over the old one.

- Your settings in `/data/adb/cmf-stereo/` are kept.
- A phone that was in Stereo is staged in Stereo again during the update.
- Settings added by the new version are added to your file automatically.

TECHNICAL

Updates are found through `updateJson` in `module.prop`, which points to `update.json` in the project repository. Your root manager compares its `versionCode` with the installed one (1090 for *v1.0.9-beta*).

The Stereo overlay lives inside the module folder, which an update replaces. That is why the installer stages Stereo again: it re-checks and re-patches your phone's own files. If they do not check out, it sets `CHANNELS=mono` and prints why: `channels : mono - stereo could not be staged:` followed by the reason.

2.11 Updating from older versions

The installer takes care of a few things left over from older versions:

- **New settings.** If your `stereo.conf` lacks a setting the new version knows, the default is added under a line `# --- added by v<version> ---`, and the installer prints `kept stereo.conf, added N new setting(s)`. Otherwise it prints `keeping existing stereo.conf`.
- **Your routing.** An `actions.conf` with real content is never overwritten (`keeping your actions.conf (N actions)`): that is your tuning.
- **The old earpiece gain mistake.** Versions before v0.9.2 shipped `Handset Volume 31`, believing it was the loudest; on this codec it is the mute value (−40 dB). Any value above 18 is corrected to 0 and the installer prints `earpiece gain corrected: 31 was -40 dB (mute), now 0 = +8 dB`.
- **The guard and the playback detector** are set again on every install on a CMF Phone 1, so phones updated from v1.0.2 or earlier get the v1.0.3 detection that also covers games.
- **The pre-release test module.** If `cmf_stereo_halpatch_test` is installed, it is marked for removal at the next reboot.

2.12 Uninstalling

Remove the module in your root manager and reboot.

- The uninstall script stops the daemon and puts back every control it touched.

- It gives the speaker amp its own gain back.
- The Stereo overlay goes with the module, so the phone is back on its stock audio files.
- `/data/adb/cmf-stereo/` is **left in place**, so a reinstall does not lose your routing and settings. Delete that folder by hand for a clean slate.

```
su
rm -rf /data/adb/cmf-stereo
```

> WARNING
>
> Only run that `rm` after the module is removed. It deletes your settings, your routing and the saved original values for good.

> TIP
>
> Uninstall the apps separately if you no longer want them; without the module they have nothing to control.

03

EVERYDAY USE

Where the controls are, what the module's card tells you, and how the module behaves day to day: with headphones, during calls, and when you turn the phone.

3.1 Where the controls are

You can control the module from five places. They all do the same thing underneath, and they stay in step.

Place	What you can do
CMF Stereo / CMF Stereo Pro app	on/off, Mono/Stereo, levels, protection settings (Pro: much more)
Quick Settings tile (from the app)	Mono / Stereo
Stereo speakers page in Nothing Settings (app with LSPosed)	the app's settings, inside Settings > Sound
Action button on the module's card	switch Mono ⇔ Stereo
Terminal: `stereoctl`	everything, including diagnostics

3.2 The module's card

Magisk, KernelSU and APatch show the module's description on its card. The module keeps the first words of that description current, so a glance tells you what is running:

Card starts with	Meaning
`[Stereo] Left and right follow the screen.`	Stereo is running.
`[Mono] Both speakers play the full mix.`	Mono is running.

`[Off] Earpiece not used. Turn on in the app or with stereoctl on.`	The module is turned off (section 3.4).
`[Stereo unavailable] This Nothing OS build is not supported yet: mono (stock audio) until a module update adds it.`	Your vendor audio files are not the analysed build, usually after a Nothing OS update (section 4.11).
`[Inactive] Not a CMF Phone 1 audio stack: nothing is changed.`	The audio files the module looks for are not there.

Two notes can follow: `Switches to stereo after a reboot.` (or mono) when a change is waiting for a reboot, and `Not running: restart it in the app.` when the daemon is not running.

> TIP
>
> Root managers read the description when the Modules page opens. Pull down on Magisk's Modules page to refresh it.

3.3 The Action button

Tap **Action** on the module's card to switch to the other mode: from Mono to Stereo, or from Stereo to Mono. The button shows what happens, for example:

```
CMF true stereo: switching to stereo

stereo applied now (audio restarted - press play again)
...
```

The rest of the output is the same as `stereoctl channels stereo` or `stereoctl channels mono` (section 6.6).

> TECHNICAL
>
> `action.sh` checks whether the Stereo overlay is staged: if it is, it asks for `mono`; if not, for `stereo`. It then runs `stereoctl channels` with that mode. It needs Magisk 28 or newer, KernelSU or APatch.

3.4 Turning the module on and off

You can turn the earpiece off without uninstalling anything: in the app, or with:

```
stereoctl off    # disarm: earpiece not used, every control put back
stereoctl on     # arm again
```

Off survives a reboot. The module stays off until you turn it on again.

> WARNING
>
> **Turning it off while in Stereo.** `stereoctl off` stops the daemon, but the Stereo files stay in place. With the daemon off, the speaker amp goes back to its stock left slot on the next stream, so the speaker would play the left channel alone. If you want the module off, switch to Mono first (`stereoctl channels mono`; it switches at once).

> TECHNICAL
>
> `off` creates the kill switch `/data/adb/cmf-stereo/disabled`, stops every daemon, restores the saved originals, and in Stereo sets the amp to its mono (L+R)/2 slot for now. `on` removes the kill switch and restarts the daemon. `ENABLED=0` in `stereo.conf` has the same effect as the kill switch.

3.5 While playing, or always on

By default the module uses the earpiece **only while audio is playing** (the app calls this While playing). The earpiece is not built to run continuously, and this saves power too.

The other choice, Always on, keeps the routing applied all the time. It is louder in theory only, harder on the earpiece, and costs idle power. The clean CMF Stereo app offers only While playing; CMF Stereo Pro and `stereo.conf` offer both.

> TECHNICAL
>
> This is the `MODE` setting: `playback` (default) or `always` (section 7.5). "Playing" is detected from the audio DSP's task switches, so games, music players and every other app count (v1.0.3 and later).

3.6 Headphones and Bluetooth

When headphones or Bluetooth audio take over, the earpiece steps aside by itself. You never have the earpiece playing to nobody while you wear headphones.

The left/right swap for turning the phone also applies only to the speakers; headphones are never swapped.

> TECHNICAL
>
> The audio HAL switches off the speaker amp (`aw_dev_0_switch`) when headphones or Bluetooth take over. The module's `guard` reads that switch once per cycle and applies the routing only while it reads `Enable` (section 7.9). A second, optional check (`REQUIRE_SPEAKER_ROUTE`, off by default) reads `dumpsys audio` instead.

3.7 Phone calls

Calls are left alone.

- In both modes, calls keep Awinic's speaker protection.
- A switch between Mono and Stereo is refused while a call is active. It is applied after the next reboot instead, or you can try again after the call.
- When the module restarts the audio service for a switch, calls are unaffected (measured on the phone).

> TECHNICAL
>
> Stereo changes only the DSP scenario `AURISYS_SCENARIO_DSP_PLAYBACK_SMARTPA`; `DSP_CALL_FINAL` keeps `smartpa_awinic`. The live switch checks `dumpsys telephony.registry` for an active call state before it does anything.

3.8 Turning the phone: left and right follow the screen

In Stereo, left and right follow how you hold the phone:

- In portrait, and sideways with the earpiece on your left, the earpiece plays the left channel and the speaker the right.
- Turned the other way, with the earpiece on your right, they swap: the earpiece plays the right channel and the speaker the left. The stereo image stays the right way round.

There is nothing to set up and no per-app list. Keep auto-rotate on. For an app with its own audio effect (such as VLC), the swap can take up to about 2 seconds after you press play or turn the phone.

> TECHNICAL
>
> Android itself swaps left and right before the audio HAL when the screen is at rotation 270, except for a track whose session has its own effect chain (VLC's Dynamics Processing). The daemon finds such tracks and names their outputs to the HAL, which swaps exactly those, only at rotation 270. Section 11.8 explains it in full.

3.9 Testing with the test files

Every release carries two test files. Play them with any music app (VLC, USB Audio Player Pro, Files and so on), with auto-rotate on.

File	What it contains	What you should hear

LEFT_ONLY_3min.wav	Sound in the left channel only.	Stereo, earpiece on your left: only the earpiece plays. Turn the phone (earpiece on your right): only the speaker plays. Mono: both play.
LEFT10s_RIGHT10s.wav	A voice says "left", then 10 s of sound from the left only; then "right" and 10 s from the right only. Six rounds.	In Stereo, each side's sound comes from the driver that is on that side.

> TECHNICAL
>
> Both files are made by tools/make-test-audio.py in the repository.

3.10 Apps that were checked

Left and right were checked to follow the rotation on the test phone with:

- ordinary apps and music players, including USB Audio Player Pro (bit-perfect mode too);
- InShot, YouTube and games;
- VLC (it uses its own audio effect, so the module swaps it; v1.0.2 and later);
- every button of the CMF L/R Test app: MUSIC, MOVIE, MOVIE + EFFECT, GAME, SPEECH, AAUDIO, AAUDIO SHARED and FLOAT.

"While playing" engages the earpiece for games and music players alike (v1.0.3). Hi-res playback (USB Audio Player Pro, Direct 192 kHz) still works in Stereo.

3.11 Nothing Settings and the apps stay in step

When something changes outside the apps, the module tells them, so the app, its Quick Settings tile and the Stereo speakers page in Nothing Settings update at once. This happens after:

- a `stereoctl` command that changes something;
- the Action button;
- the module's own switch back to Mono (its crash-loop safety net);
- once each time the daemon starts, after boot.

If an app is not installed, nothing happens; it is silent and runs in the background.

> TECHNICAL
>
> The module calls `content call --uri content://<app>.settingstile --method stateChanged` for `com.tiimoo.cmfstereo` and `com.tiimoo.cmfstereo.clean`. Commands that notify: `on`, `off`, `reset`, `solo`, `unsolo`, `restart`, and `guard`, `protect` and `channels` when given a value. This is tested with a stand-in `content` command; it has not yet been seen working on a phone.

3.12 Battery and background work

The daemon checks every 2 seconds whether something is playing, and keeps the routing in place while it is. When nothing plays, it does very little.

If you notice battery drain, you can raise the interval to 5 seconds (`WATCH_INTERVAL`, section 7.6). The audio system resets mixer controls on every

route change, so the interval cannot be very large.

> TECHNICAL
>
> Work kept small on purpose: one mixer read per cycle for the guard (no `dumpsys`); `tinymix`'s calling style detected once per process; the amp's attenuation checked when playback starts and every 10th cycle (the Awinic driver writes a kernel log line for each read); `dumpsys media.audio_flinger` read only while playing at rotation 270.



MONO AND STEREO

The two channel modes in detail: what each sounds like, how to switch, what happens during a switch, and exactly what Stereo changes on your phone.

4.1 Mono

Mono is the default, and it is stock audio. Both the earpiece and the bottom speaker play the same full mix. The result is louder, fuller and taller than the single bottom-firing speaker.

- The phone's audio files are untouched.
- The speaker keeps Awinic's speaker protection.
- The earpiece plays without the module's earpiece limiter (that limiter runs only in Stereo).

This was the whole module once, when it was called cmf-fake-stereo.

4.2 Stereo

Stereo is real left and right. The earpiece and the speaker each play one channel, following the screen (section 3.8).

It costs Awinic's speaker-protection processing on media. In its place, the module's own protection runs: a fixed amp attenuation, a deep bass guard and an earpiece limiter, measured against what Awinic does, with an optional speaker boost and earpiece leveller. Chapter 5 covers each one.

> WARNING
>
> Stereo has no temperature model of the speaker. For long, loud listening, use Mono.

4.3 Which mode to choose

You want	Choose
Games, films and music with real sides	Stereo
Long, loud listening	Mono
The fullest Awinic speaker protection	Mono
The module off, with no change to the speaker	Mono, then `stereoctl off`

You can change your mind at any time; switching takes about a second.

4.4 Switching between Mono and Stereo

Use any of these. Each switches at once, without a reboot:

- the **Mono / Stereo** choice in CMF Stereo or CMF Stereo Pro;
- the apps' **Quick Settings tile**, or their Stereo speakers page in Nothing Settings;
- the **Action** button on the module's card;
- the terminal:

```
stereoctl channels          # what is running, and what is staged
stereoctl channels stereo    # switch to stereo now
stereoctl channels mono      # back to stock now
```

Your choice is remembered (`CHANNELS` in `stereo.conf`) and used at every boot.

> TIP
>
> Change the mode with these, not by editing CHANNELS by hand: Stereo needs its files prepared and checked first, which only the switch does.

4.5 What you hear during a switch

The module restarts only the phone's audio service, not the phone:

- about a second of silence;
- players pause: press play again;
- calls are unaffected.

The output says `stereo applied now (audio restarted - press play again)` (or `mono applied now …`). If you ask for the mode that is already running, it says `stereo (already running)` and changes nothing.

4.6 When a switch waits for a reboot

The module only switches live when every check passes. If one does not, it undoes what it did and the change waits for the next reboot. You then see, for example:

```
stereo staged. Reboot to apply (a call is active).
```

Reasons you may see in the brackets:

Reason	What to do
a call is active	Try again after the call, or reboot.
a mount over the audio files is not ours - not touching it	Another module mounts over the same files. Reboot; see section 9.9 about other modules.
no verified stereo copy of …	The prepared files did not check out. Reboot, or switch again.
… does not read back as the stereo file / cannot mount over …	A file did not land as expected. Reboot.
the audio HAL did not come back on the stereo files	The module went back to the stock files. Reboot.

While a change waits, `stereoctl channels` shows `staged: stereo (reboot to apply)` and the card adds `Switches to stereo after a reboot`.

4.7 What a live switch checks

A live switch is careful by design. In short: it only removes its own changes, puts on exactly what the wanted mode needs, checks every file where the audio service reads it, and makes sure the audio service is still running a few seconds later.

TECHNICAL

- Magisk bind-mounts each of the module's files over its vendor path at boot, in init's mount namespace, which the audio HAL shares (measured).
- A live switch takes off **only its own** mounts. If any mount from anywhere else sits on those paths, the switch is refused.
- It then mounts what the wanted mode needs: the staged files for Stereo, nothing for Mono, or, where Magisk left an empty placeholder rather than the stock file, a copy of the phone's own stock files.
- That stock copy is made when the phone booted Mono, or rebuilt from the Stereo files by putting the original bytes back, and kept only if it hashes to the stock build.
- Every file is read back through `/proc/1/root` and must hash to the expected build.
- The audio HAL is restarted with `setprop ctl.restart vendor.audio-hal`. The new process must be running, and still be the same process **3 seconds later**.
- If anything fails, the mounts are undone and the HAL is restarted only once all three files read back as stock. If they cannot be put back, the running HAL is left alone and a reboot puts everything right.

4.8 What Stereo changes, and nothing else

Stereo changes five things. Everything else on the phone stays as it is.

1. **The audio HAL's fold to mono is switched off.** In its place the HAL loads the module's protection library and runs it on every buffer, and the audio DSP is told the speaker is stereo.

2. **The audio DSP's fold to mono**: Awinic's speaker protection for media is replaced by MediaTek's pass-through, which the phone's own configuration file already declares. Calls keep Awinic.
3. **The amp's slot**: while the earpiece plays, the speaker amp plays the right slot; whenever the earpiece is not playing, the amp plays a mono mix, so the speaker never plays one channel alone.
4. **Attenuation**: the amp is held 6 dB down by default (the speaker level).
5. **Protection**: the module's library guards the speaker's deep bass and limits the earpiece (chapter 5).

> TECHNICAL
>
> 1. `AudioALSAPlaybackHandlerBase::doStereoToMonoConversionIfNeed` in `audio.primary.mt6878.so` (and `audio.primary.mediatek.so`) is patched (`hal-swap/patch.S`) to load `libcmfstereo.so` and call it; the DSP task gets `hw_info` bit `0x4` (two branches). Four places in all; the stock bytes every earlier version changed are written back first, and the result must hash to the tested build.
>
> 2. `AURISYS_SCENARIO_DSP_PLAYBACK_SMARTPA` in `aurisys_config_rv.xml` runs `aurisys_demo` instead of `smartpa_awinic`. The phone's own file already declares `aurisys_demo` for that scenario (2 channels in and out, IV reference).
>
> 3–4. The daemon writes the amp's I2S slot register and holds `aw_dev_0_volume` (sections 4.9 and 5.3). Mono gives the amp its previous value back.

4.9 The first second of a stream

In Stereo, the very first second or two of a new stream can come from the speaker as the stock left slot, before the module puts the amp on its right slot. This is expected.

> TECHNICAL
>
> The Awinic driver rewrites its slot register from its profile on every stream start. The daemon re-asserts the slot on its next cycle (every `WATCH_INTERVAL` seconds, 2 by default). Register `0x06` bits 11:10: `01` left (stock `0x34e8`), `10` right (`0x38e8`), `11` mono (L+R)/2. When nothing plays, the daemon does not touch the amp at all, so its I2C bus is not woken.

4.10 No vendor file ships in this module

The module carries no file from Nothing or MediaTek. To prepare Stereo (`staging`), it copies your phone's own audio files, checks they are exactly the analysed build, patches them, and checks the result against the tested one, before putting it in the module's own overlay folder with the original SELinux labels.

Anything else — another firmware, a different file — is refused, and the phone stays on Mono.

TECHNICAL

Staged files: /vendor/lib64/hw/audio.primary.mt6878.so, /vendor/lib64/hw/audio.primary.mediatek.so, /vendor/etc/aurisys_config_rv.xml, plus the module's own libcmfstereo.so (checked against its own hash). They go into system/vendor/... inside the module folder; the vendor partition is never written.

4.11 Stereo after a Nothing OS update

A Nothing OS update can replace the audio files Stereo was built from. The module checks for this at every boot, before anything is mounted. If the files differ:

- the Stereo overlay is removed and the phone boots on stock Mono;
- the card says [Stereo unavailable], and stereoctl status shows build: updated;
- Stereo stays refused until a module release supports the new build.

The CMF Phone 1 is expected to get security updates only, but those can still replace the audio files. When the new build has been checked, a module update adds it.

TIP

This guard has been tested on fake vendor files only; it will first be seen on a phone with the next Nothing OS update.

4.12 Known limits in Stereo

- **Two apps at once.** If an app with its own audio effect (VLC) and an ordinary app play on the same output at the same time, with the earpiece on your right, one of them comes out reversed. The audio HAL sees only the mix.
- **Compressed offload** (the DSP decoding MP3 itself) is refused by Android on this phone, so apps never use it here. The module does not swap compressed audio.
- **Protection after a live switch from a Mono boot** starts only after a reboot (section 5.8).
- **The first second of a stream** may play on the stock left slot (section 4.9).
- **The earpiece will not match the speaker.** It is a receiver; balance the two by turning the speaker down (section 5.10).

> TECHNICAL
>
> Offload: `AudioManager.isOffloadedPlaybackSupported` is false for MP3 on the speaker. The likely reason, not yet proven, is that the audio policy refuses offload while a non-offloadable effect is on, and Nothing's Dirac is on for the output mix.

05

PROTECTION AND LEVELS

Stereo replaces Awinic's speaker protection with the module's own. This chapter explains each part, every setting, how loud each driver can go, and the risks, plainly.

5.1 Why Stereo needs its own protection

In Mono, Awinic's software on the audio DSP protects the speaker. It works from a live model of the speaker, keeping it within its movement and temperature limits. But that same software mixes left and right into one, which is exactly what Stereo has to avoid. So Stereo turns it off for media, and the module protects the speaker and earpiece itself.

The module's protection was built from measurements of what Awinic does (CMF L/R Test, DRIVER STUDY). Measured in Stereo without it, the speaker got up to about **10 dB more deep bass** than Awinic lets through, and the earpiece **clipped** in its loudest 1.5 dB. The protection holds both.

It runs inside the audio HAL, for every app, before the sound reaches the DSP.

5.2 Risks, plainly

WARNING

The speaker in Stereo. In Mono, Awinic keeps the speaker within its movement and temperature limits from a live model of it. In Stereo, media gets:

- the amp's own hardware over-temperature and over-current cut-outs;
- the speaker's peaks held **6 dB below the amp's full output** at speaker level 10, 2 dB more per level down;
- its deepest bass held back when it gets loud (the deep bass guard).

Both were measured against Awinic's, but they are **fixed limits**: there is **no temperature model.** Speaker boost makes it louder; a limiter then holds its peaks at Awinic's own level, about 5 dB below full output. For long, loud listening use **Mono.** In Stereo, lower the speaker level and back off at the first distortion.

WARNING

The earpiece. In Stereo the earpiece limiter (on by default) holds it at −2.5 dBFS, below where it starts to crackle, and the leveller never lifts its peaks above that. In Mono the earpiece plays without protection. It has no protection circuit of its own: back off at the first buzz or rattle.

How the speaker holds up to long, loud Stereo listening has not been tested yet.

5.3 The speaker level and the 6 dB margin

In Stereo the speaker amp is held a fixed amount below its full output. This margin is part of what stands in for Awinic's protection.

App speaker level	Amp held down by
10 (loudest)	6 dB (default)
9	8 dB
8	10 dB
7	12 dB
6	14 dB
5	16 dB
4	18 dB
3	20 dB
2	22 dB
1	24 dB

CMF Stereo never goes below the 6 dB margin. Raise the attenuation if you play loud.

> WARNING
>
> CMF Stereo Pro and stereo.conf accept values under 6 dB (the setting's range is 0–30). Pro asks before going under 6 dB. Below that margin the speaker has less headroom than the module was designed around; with the protection library not running (after a live switch from a Mono boot, until a reboot), only the amp's own cut-outs protect it.

> TECHNICAL
>
> Setting: STEREO_ATTEN_DB (default 6, range 0–30). It is applied through the amp's own volume control, aw_dev_0_volume, an attenuation in 1/8 dB steps (0–720 = 0–90 dB, per the aw882xx driver), less the speaker boost while the protection runs. A larger attenuation set by someone else (you, or the driver's own monitor) is kept. Mono gives the amp its previous value back.

5.4 Speaker boost

Speaker boost makes the speaker louder in Stereo by taking up to 6 dB off the fixed attenuation. A limiter then keeps the speaker's loudest moments where Awinic keeps them in Mono, about 5 dB below the amp's full output.

Values	Default	Set with
0 to 6 (dB)	0	the apps, or stereoctl protect boost N

- At 0 the speaker sounds as before.

- It works only while the protection runs. If the protection is not running, the boost is ignored and the full attenuation is kept.

```
stereoctl protect boost 3
```

WARNING

Boost raises the average level, and so the heat in the speaker. The peak limiter does not model temperature. Boost 6 has been tried only briefly so far. For long listening, keep boost low or use Mono.

TECHNICAL

Amp attenuation = STEREO_ATTEN_DB − SPEAKER_BOOST (never below 0). The speaker's peak ceiling is (attenuation − 5) dB of full scale when that is negative, so that peaks never exceed −5 dB below the amp's full output; at an attenuation of 5 dB or more the ceiling is off (0), because the attenuation alone already holds them there.

5.5 Deep bass guard

The deep bass guard holds back only the speaker's deepest bass, below 100 Hz, and only when it gets loud. Quiet bass is untouched, unlike Awinic, which always cuts it.

Value	Deep bass held at, below the amp's full output
off	not held
light	−8 dB

medium (default)	−14 dB
strong	−20 dB

```
stereoctl protect bass medium
```

For comparison, Awinic in Mono cuts 100 Hz by 16 dB at every level.

> **WARNING**
>
> Turning the guard off removes the one protection aimed at the speaker's excursion in Stereo. Leave it on unless you know why you are turning it off.

> **TECHNICAL**
>
> A 4th-order Linkwitz-Riley split at 100 Hz on the speaker channel; the band below is limited only when it exceeds its ceiling. The ceiling handed to the library is (level + attenuation) dB of the samples' full scale, so it follows the speaker level: for example speaker level 5 (16 dB) with strong gives −4 dBFS on the signal, −20 dB below the amp's full output (verified on the phone).

5.6 Earpiece limiter

The earpiece limiter stops the earpiece clipping at its very loudest. It holds the earpiece at −2.5 dBFS; above that, the earpiece crackles. Its bass is kept.

Values	Default	Set with

on, off	on	the apps, or stereoctl protect earpiece on|off

```
stereoctl protect earpiece on
```

> WARNING
>
> Keep it on. Measured on the phone, the earpiece clips in its top 1.5 dB of full scale: odd harmonics jump to 25–35 % at 150 Hz, 200 Hz and 2 kHz. The limiter runs only in Stereo.

5.7 Earpiece leveller

The leveller is optional. It brings up the earpiece's quiet and medium parts, so the earpiece side keeps up with the speaker. You will notice it most with films, classical music and older recordings; loud modern pop is already levelled in the studio and changes little.

Value	Quietest parts raised by at most
off (default)	—
light	4 dB
medium	8 dB
strong	12 dB

- Loud parts, bass and every frequency stay as they are.

- The earpiece's peaks never go above what it already gets at full volume.
- Silence stays silent: the hiss between tracks is not raised.

```
stereoctl protect leveller medium
```

On the phone, the leveller made a clear difference on the earpiece side, with no distortion heard.

> TECHNICAL
>
> It raises loudness towards −18 dBFS, making up 35 / 55 / 75 % of the gap (light / medium / strong) up to the 4 / 8 / 12 dB limit, and leaves anything below −60 dBFS alone. Peaks stay at the earpiece's ceiling: −2.5 dBFS with the limiter on, −0.1 dBFS with it off.

5.8 When the protection runs

The protection runs only in Stereo. Check it with:

```
stereoctl protect
```

The first line tells you:

It says	Meaning
`protection: running`	The audio HAL loaded the library and it is working.

`protection: not running yet (...)`	Stereo is on, but the library has not started: it starts with the first sound on the speaker.
`protection: off (mono)`	Mono is running; the protection is not used.

> WARNING
>
> After a live switch from a Mono boot, the protection starts only after the next reboot, because the library is not in place until then. Until you reboot, Stereo plays with the fixed attenuation alone and the boost is ignored. Reboot once after you first switch to Stereo.

> TECHNICAL
>
> The library marks itself with `vendor.audiohal.cmf_prot=1` on its first speaker buffer; the module also checks that the file the HAL loads hashes to the tested build. Settings reach the library within one daemon cycle (and at once if Stereo runs when you change them).

5.9 The earpiece level

By default the earpiece plays at the receiver amp's maximum, +8 dB, the same value the audio HAL uses when it drives the earpiece itself.

In CMF Stereo, **earpiece level** 10 is that maximum, and each step down is 2 dB quieter, to level 1 at −10 dB. In CMF Stereo Pro you set it in dB.

By hand, change the number in `actions.conf`:

```
ctl|Handset Volume|0
```

Raise the number (towards 18) to make the earpiece quieter.

> TECHNICAL
>
> `Handset Volume` is the receiver amp's gain on the MT6369 codec. Read from the codec driver (`snd-soc-mt6369.ko`), the range is 0–18 and the register runs the opposite way to the dB scale ALSA advertises:
>
> | Value | Gain |
> |---|---|
> | 0 | +8 dB (loudest) |
> | 8 | 0 dB |
> | 18 | −10 dB |
> | 31 | −40 dB (the codec's mute value) |
>
> Measured: the earpiece's whole path is already at full level; there is no hidden attenuation to win back. `ADDA_DL_GAIN`, the digital gain before the DAC, is left alone: the HAL parks it at 63311 of 65535 (−0.3 dB), and `ADDA_DL` is shared with the headphone path. `ADDA_DL_MAX_VOL` was tried for a louder earpiece: no clear difference, left out.

5.10 Balancing the earpiece and the speaker

The earpiece is a receiver; it will never match the speaker. To balance the two:

- turn the speaker down (the apps' speaker level, or `STEREO_ATTEN_DB`);

- use the earpiece leveller to bring up the earpiece's quiet parts without raising its peaks.

> WARNING
>
> Do not try to balance by pushing the earpiece harder. It is already at its maximum, and it has no protection circuit of its own.

5.11 Awinic compared with the module

Measured with CMF L/R Test DRIVER STUDY: tones from 100 Hz to 8 kHz at −30 to −3 dBFS, with the HAL's and the DSP's own recordings of what reached them and the HAL's raw 24-bit two-microphone capture.

	Awinic (Mono, stock)	The module in Stereo
Speaker peaks	about −5 dB below the amp's full output	about −6 dB (fixed attenuation); with boost, a limiter keeps them at −5 dB
100 Hz	cut by 16 dB at every level	untouched while quiet; held at −8 / −14 / −20 dB below full output when loud (light / medium / strong)
150 Hz – 3 kHz	about +3.5 dB EQ	unchanged
Earpiece	(not Awinic's) clips in its top 1.5 dB of full scale	limited at −2.5 dBFS; optional leveller raises quiet passages, peaks unchanged

The limiters add at most 1.2 % distortion while working (at 50 Hz), under 0.1 % above 1 kHz. Nothing changes while a signal is below its ceiling.

5.12 Choosing your settings

The defaults are a sensible start: speaker level 10 (6 dB), boost 0, bass guard medium, earpiece limiter on, leveller off. From there, the module's own advice is:

- **Loud, long listening:** switch to Mono.
- **Playing loud in Stereo:** raise the attenuation (lower the speaker level).
- **At the first distortion:** back off: lower the speaker level, reduce boost.
- **Earpiece too quiet next to the speaker:** turn the speaker down, or try the leveller.
- **Earpiece buzzes or rattles:** keep the limiter on and make the earpiece quieter (a higher Handset Volume number).

> TIP
> Which Stereo speaker level sounds as loud as stock Mono has not been measured yet; compare by ear with the same track.

06

THE COMMAND LINE: STEREOCTL

Every feature of the module is reachable from one command, `stereoctl`. This chapter lists every command, what it does, what it prints, and an example.

6.1 Opening a root shell

Run `stereoctl` as root, from Termux or from a computer with `adb`:

```
# in Termux, or after: adb shell
su
stereoctl status
```

Or run one command at a time:

```
su -c stereoctl channels stereo
```

`stereoctl` with no command is the same as `stereoctl status`. `stereoctl help` (or `-h`, `--help`) prints the list of commands.

> TECHNICAL
>
> `/system/bin/stereoctl` is a small shim from the module that runs `/data/adb/modules/cmf_stereo/scripts/stereoctl`. An unknown command prints `unknown command: …` and the list.

6.2 All commands at a glance

Command	What it does
`status [brief]`	module, daemon and control state
`doctor`	diagnose why the earpiece is silent

channels [mono|stereo]	show or choose the speaker channels (applied at once)
protect	Stereo protection settings, and whether it runs
protect boost 0-6	speaker boost
protect bass off|light|medium|strong	deep bass guard
protect earpiece on|off	earpiece limiter (−2.5 dBFS)
protect leveller off|light|medium|strong	earpiece leveller
on | off	arm / disarm (off survives a reboot)
solo | unsolo	speaker off, earpiece only / back to normal
guard on|off	re-enable / bypass the speaker-follows guard
apply | revert	one-shot apply / restore of the routing
restart	restart the daemon
reset	clear saved state and start clean
log [n]	show the end of the log
edit	print the config file paths
probe [dir]	dump the audio hardware
report	fold the newest probe into one shareable .txt
scan [pattern]	search the mixer for candidate controls
ctl NAME [VALUE]	read or write one mixer control
dump	every mixer control and its value

`diff [seconds]`	snapshot the mixer, wait, show what changed
`xml-patch` | `xml-revert`	install / remove the stereo speaker-port overlay (reboot)

6.3 status

Shows the module, the daemon, and the live value of every control the module manages. It is the first thing to run when something seems wrong.

```
stereoctl status
```

```
module:    /data/adb/modules/cmf_stereo
config:    /data/adb/cmf-stereo/stereo.conf
actions:   /data/adb/cmf-stereo/actions.conf (3 configured)
armed:     yes
mode:      playback (interval 2s)
daemon:    running (pid <pid>)
tinymix:   <path> (<style> style)
playback:  active
boot:      0 strike(s) since last successful boot
channels:  stereo
build:     known
amp slot:  right   attenuation: <steps>/8 dB (holds 6 dB)
xml overlay: 0 file(s)

current values:
  ctl    Handset Volume            want=0         now=0
  ctl    ADDA_DL_CH1 DL_24CH_CH1    want=1         now=1
  ctl    RCV Mux                   want=Voice Playback now=Voice Playback
```

Line	What to look for
armed	no (disabled) means the module is off: stereoctl on.
daemon	not running: stereoctl restart.
tinymix	NOT FOUND or UNUSABLE: mixer actions cannot run (section 9.13).
playback	idle when nothing plays; the routing then is not applied (with MODE=playback).
boot	Failed boots counted by the watchdog; 0 is normal.
channels	What runs; a waiting change shows as mono (switches to stereo after reboot).
build	known (supported), updated (Nothing OS update, Stereo unavailable), absent (not a CMF Phone 1 audio stack).
amp slot	Stereo only: right while the earpiece plays, mono otherwise; the attenuation the amp holds.
current values	With audio playing, want= should match now= for each control.

6.4 status brief

The same as status, but it stops after the build and amp slot lines and does not read every control back. The apps use it to refresh quickly after each tap.

```
stereoctl status brief
```

6.5 doctor

Diagnoses why the earpiece is silent. It walks every layer that can silence the earpiece, in the order they apply, and names the one responsible. Start some music first, then run:

```
stereoctl doctor
```

It checks six things:

1. **module armed** — the kill switch is not set;
2. **daemon** — exactly one daemon is running;
3. **tinymix** — the mixer tool works;
4. **playback** — audio is detected as playing, and by which detector;
5. **guard** — the speaker amp switch allows the routing;
6. **applying now** — it applies the routing and reads each control back: `ok` or `FAILED`.

Each problem comes with its fix, for example `NO - kill switch set (it survives reboots). Fix: stereoctl on`. Section 9.3 explains every message.

> **TIP**
> If nothing is playing, doctor skips step 6 on purpose: applying would switch the earpiece on outside normal use and make it look as if doctor fixed something.

6.6 channels

Shows or chooses the speaker channels. A change is applied at once by restarting the audio HAL; if that cannot be done safely, after a reboot.

```
stereoctl channels
```

```
running: mono
staged:  mono
```

```
stereoctl channels stereo
```

```
stereo applied now (audio restarted - press play again)

  left and right play on the earpiece and the speaker, following the screen.
  Awinic's speaker protection is off for media in this mode; the
  amp is held 6 dB down instead (STEREO_ATTEN_DB), and the
  module's own protection holds the deep bass and the earpiece
  ('stereoctl protect'; after a live switch from a mono boot it starts
  with the next reboot).
  Back off at the first sign of distortion, and use
  'stereoctl channels mono' for loud listening.

  Turned the other way (earpiece on your right), left and right
  swap with the screen, so the image stays the right way round.
```

```
stereoctl channels mono
```

```
mono applied now (audio restarted - press play again)
```

Other results: `stereo (already running)`, `stereo staged. Reboot to apply (<reason>).` (section 4.6), and `stereo NOT staged: <reason>` when your phone's files cannot be prepared (section 9.5).

> TECHNICAL
>
> Stereo: saves a checked stock copy, stages the overlay, sets `CHANNELS=stereo`, then switches live. Mono: from a Stereo boot rebuilds the stock files first, removes the overlay, sets `CHANNELS=mono`, switches live, and gives the amp its own gain and left slot back. With a mode given, it also updates the card and tells the apps.

6.7 protect

Shows the Stereo protection settings and whether the protection runs.

```
stereoctl protect
```

```
protection:  running
boost:       0 dB  (amp attenuation now 6 dB)
bass:        medium
earpiece:    on
leveller:    off
ceilings:    speaker 0, bass -800, earpiece -250 (1/100 dB of full scale, 0 = off), leveller 0
```

The example shows the default settings with the protection running. The ceilings line is what the module hands the library, in hundredths of a dB of full scale; 0 switches a stage off. With the defaults the speaker's own peak limiter is off (the 6 dB attenuation already holds the peaks), the bass guard is at −8 dBFS on the signal (−14 dB below the amp's full output), and the earpiece at −2.5 dBFS.

Each protect sub-command below saves its value in stereo.conf and then prints this same summary.

6.8 protect boost

Sets the speaker boost, 0 to 6 dB less amp attenuation, only while the protection runs. The speaker's peaks stay at Awinic's level, about −5 dB.

```
stereoctl protect boost 2
```

Any value outside 0–6 prints usage: stereoctl protect boost 0-6 and changes nothing. See section 5.4 for the warning.

6.9 protect bass

Sets the deep bass guard: off, light, medium or strong. It holds the speaker's bass below 100 Hz only when it gets loud, at −8 / −14 / −20 dB below the amp's full output.

```
stereoctl protect bass strong
```

6.10 protect earpiece

Turns the earpiece limiter on or off. On, it holds the earpiece below −2.5 dBFS, where it would otherwise clip.

```
stereoctl protect earpiece on
```

6.11 protect leveller

Sets the earpiece leveller: off, light, medium or strong. It raises the earpiece's quiet and medium passages by at most 4 / 8 / 12 dB, with no frequency changed and peaks never higher.

```
stereoctl protect leveller light
```

6.12 on and off

off disarms the module: it stops the daemon and puts every control back. It survives a reboot.

```
stereoctl off
```

```
daemon stopped
disarmed and reverted - controls are yours until 'stereoctl on'
note: this survives a reboot. 'stereoctl on' to re-arm.
```

In Stereo it adds a note: the Stereo overlay is still mounted, the amp will return to its left slot on the next stream, and you should run `stereoctl channels mono` for stock audio.

`on` arms it again and restarts the daemon:

```
stereoctl on
```

```
armed
daemon restarted (pid <pid>)
```

6.13 solo and unsolo

`solo` turns the speaker amp off so you hear the earpiece alone. Use it to audition the earpiece, or to prove it is really playing. `unsolo` puts the speaker back.

```
stereoctl solo
```

```
speaker off (aw_dev_0_switch was Enable), guard bypassed
earpiece only - run 'stereoctl unsolo' when done
```

```
stereoctl unsolo
```

```
speaker restored to Enable, guard active
```

> TIP
>
> Solo bypasses the guard first. Otherwise turning the speaker amp off would also silence the earpiece, because the guard reads that as "headphones took over".

> WARNING
>
> In solo the earpiece plays everything on its own. Keep the volume moderate and always finish with unsolo.

6.14 guard

The guard makes the earpiece follow the speaker: it plays only while the speaker amp is on. You can bypass it or turn it back on:

```
stereoctl guard        # show: guard: aw_dev_0_switch = Enable  (or: guard: bypassed)
stereoctl guard off    # guard bypassed
stereoctl guard on     # guard active
```

> **WARNING**
>
> With the guard bypassed, the earpiece can play while headphones or Bluetooth are in use. Turn it back on when you are done.

6.15 apply and revert

One-shot commands for testing. `apply` writes every action in `actions.conf` once; `revert` puts back the saved originals.

```
stereoctl apply     # prints: applied
stereoctl revert    # prints: reverted
```

> **TIP**
>
> While the daemon runs it keeps doing its own applying and reverting every cycle, so a one-shot change may be undone within seconds. For hand testing, turn the module `off` first.

6.16 restart

Stops every running daemon and starts one fresh, with no start-up delay. Use it after editing `stereo.conf`, or when the daemon is not running.

```
stereoctl restart
```

```
daemon restarted (pid <pid>)
```

If it fails: `daemon did not start - check: stereoctl log`.

6.17 reset

Clears the remembered state and starts clean: the saved original values, the guard bypass, the solo state and the kill switch. Then it restarts the daemon.

```
stereoctl reset
```

```
cleared: saved originals, guard bypass, solo state, kill switch
daemon restarted (pid <pid>)
```

> TIP
>
> Use it when a revert keeps restoring a wrong value: saved originals survive a reinstall, and a bad one makes every revert restore it. Note that reset also clears the kill switch, so a module you had turned off is on again.

6.18 log

Shows the end of the module's log, 50 lines by default, or as many as you ask for.

```
stereoctl log
stereoctl log 200
```

Lines look like this:

```
<date> <time> [info] daemon started (mode=playback interval=2s actions=3 channels=stereo)
<date> <time> [info] stereo engaged
<date> <time> [info] playback stopped - reverted
```

If there is none yet: `no log yet.`

> TECHNICAL
>
> The log is `/data/adb/cmf-stereo/stereo.log`. It is moved to `stereo.log.1` when it passes `LOG_MAX_KB` (512 KB by default), when the daemon starts. How much it writes is set by `LOG_LEVEL`.

6.19 edit

Prints the paths of the two config files, so you can open them in an editor.

```
stereoctl edit
```

```
/data/adb/cmf-stereo/stereo.conf
/data/adb/cmf-stereo/actions.conf
```

6.20 probe

Dumps the phone's audio hardware into a folder on your storage, for working out the routing on a new device. It changes nothing on the phone.

```
stereoctl probe            # writes to /sdcard
stereoctl probe /some/dir    # or to another folder
```

It shows its progress in eleven steps, then a summary:

```
Probing audio hardware -> /sdcard/cmf-stereo-probe-<timestamp>
  [1/11] device identity
  [2/11] ALSA topology
  [3/11] mixer controls
  ...
  [11/11] safety baseline

Done. Report: /sdcard/cmf-stereo-probe-<timestamp>
      Tarball: /sdcard/cmf-stereo-probe-<timestamp>.tar.gz
```

The folder holds the device identity, the ALSA topology, every mixer control, candidate control names, the smart-amp drivers, the HAL config files, the speaker device port, the HAL implementation, the current routing, MediaTek's AudioParam tree and a safety baseline of the earpiece gains. Section 12.8 lists every file.

6.21 report

Folds the newest probe into one plain text file, because chat and mail clients often refuse .tar.gz files.

```
stereoctl report
```

```
wrote /sdcard/cmf-stereo-report.txt (<size>, <n> lines)
source: /sdcard/cmf-stereo-probe-<timestamp>
```

If no probe exists yet: no probe report found - run: stereoctl probe.

> TECHNICAL
>
> It includes SUMMARY.txt, 00-device.txt, 01-alsa.txt, 02-mixer.txt, 03-candidates.txt, 04-amplifier.txt, 06-speaker-port.txt, 07-hal.txt, 10-safety.txt, 09-audio-param/index.txt and the audio policy XML files. The command also accepts a probe folder as its argument instead of the newest one.

6.22 scan

Searches the mixer for candidate controls by name. With no pattern it looks for receiver, earpiece, speaker, mux and amp names.

```
stereoctl scan
stereoctl scan 'ADDA_DL'
```

> TECHNICAL
>
> The pattern is a case-insensitive extended regular expression; the default in the code is `receiver|earpiece|rcv|handset|spk|speaker|mux|amp`.

6.23 ctl

Reads or writes a single mixer control. This is the one to experiment with: find a candidate with `scan`, flip it while music plays, and listen to the earpiece.

```
stereoctl ctl 'Handset Volume'
```

```
Handset Volume = 0
  (<raw line, with the range or the enum options>)
```

```
stereoctl ctl 'Handset Volume' 4
```

```
Handset Volume: 0 -> 4
```

Errors: `no such control: NAME`, `failed to set 'NAME'`, or `usage: stereoctl ctl NAME [VALUE]` with no name.

> WARNING
>
> A wrong write can mean silence, a crash of the audio HAL, or an overdriven earpiece. Change gains one step at a time, and turn the module off first if you test by hand, or the daemon will undo your change within seconds.

6.24 dump

Prints every mixer control and its value.

```
stereoctl dump > /sdcard/mixer.txt
```

6.25 diff

Takes a snapshot of the mixer, waits, takes another, and shows what changed. Start or stop playback, or plug in headphones, during the wait: this is how you find which controls really carry media.

```
stereoctl diff 20
```

```
snapshot taken (<n> controls).
now change what you want to observe - start music, stop it, plug in
headphones - you have 20s.

controls that changed:
  - <control before>
  + <control after>
```

The default wait is 20 seconds. If nothing changed: (no control changed).

6.26 xml-patch and xml-revert

An optional tool for other devices. If a phone's audio policy declares its speaker as mono, Android mixes to one channel before the audio HAL sees the sound. xml-patch widens only the speaker's port to stereo, in an overlay copy; xml-revert removes it. Both take effect after a reboot.

```
stereoctl xml-patch
stereoctl xml-revert
```

On the CMF Phone 1 it does nothing: its policy already declares a stereo speaker port, so you will see:

```
  skip (already stereo): <file>

Nothing patched - the policy does not declare a mono speaker port,
so this step is not what is holding stereo back on this device.
```

TECHNICAL

Only the speaker <devicePort> block is rewritten (AUDIO_CHANNEL_OUT_MONO to AUDIO_CHANNEL_OUT_STEREO); everything else stays byte-identical. The result must pass the structural check (section 8.7) or it is rejected. The overlay is guarded by the boot watchdog. Alone it does not create stereo; it only stops the framework folding to one channel.

07

CONFIGURATION FILES

All settings live in two text files that survive module updates. This chapter describes every setting: its values, its default, what it does and when to change it.

7.1 Where the files live

Everything is in `/data/adb/cmf-stereo/`, which survives module updates and uninstalls.

File	What it holds
`stereo.conf`	behaviour: mode, channels, attenuation, protection, poll interval, log level
`actions.conf`	the routing itself, one action per line
`stereo.log`	what the daemon did
`state/`	saved original values, used to revert

Section 12.10 lists every other file the module keeps there.

7.2 Editing the settings safely

- Prefer the apps or `stereoctl`: `stereoctl channels` for the mode and `stereoctl protect` for the protection. They write the file for you and apply the change.
- To edit by hand, open the file as root with any text editor (`stereoctl edit` prints the paths), save it, then run `stereoctl restart`.
- `MODE` and `ENABLED` are re-read every cycle; they need no restart.
- Each line is `NAME=value`. Lines starting with `#` are comments.

> **WARNING**
>
> Do not set `CHANNELS=stereo` by hand: Stereo needs its files staged and checked, which only `stereoctl channels stereo`, the apps or the Action button do.

> **TECHNICAL**
>
> The file is read by the shell (it is sourced), so keep values simple: no spaces around `=`, and quote values that contain `|`, as the installer does for `PLAYBACK_CTL`.

7.3 All settings at a glance

Setting	Values	Default
`ENABLED`	1, 0	1
`MODE`	playback, always	playback
`WATCH_INTERVAL`	seconds	2
`STARTUP_DELAY`	seconds	10
`REQUIRE_SPEAKER_ROUTE`	0, 1	0
`GUARD_CTL` / `GUARD_VALUE`	mixer control / value	empty (CMF Phone 1: `aw_dev_0_switch` / `Enable`)
`PLAYBACK_CTL` / `PLAYBACK_VALUE`	controls separated by `|` / value	empty / 1 (CMF Phone 1: six DSP task switches / 1)
`MIXER_CARD`	ALSA card number	empty (card 0)

CHANNELS	mono, stereo	mono
STEREO_ATTEN_DB	0–30 (dB)	6
SPEAKER_BOOST	0–6 (dB)	0
BASS_PROTECT	off, light, medium, strong	medium
EARPIECE_LIMIT	on, off	on
EARPIECE_LEVELLER	off, light, medium, strong	off
LOG_LEVEL	debug, info, warn	info
LOG_MAX_KB	kilobytes	512

7.4 ENABLED

The master switch. 1 (default): the module works. 0: it does nothing at all.

When to change it: rarely; stereoctl off and the apps' on/off do the same through a separate kill-switch file. Re-read every cycle.

> WARNING
>
> As with stereoctl off: in Stereo, switch to Mono first, or the speaker may play the left channel alone once the amp returns to its stock slot.

7.5 MODE

Value	Meaning
`playback` (default)	Drive the earpiece only while audio is actually playing. Recommended: the earpiece is not built to run continuously.
`always`	Keep the routing applied at all times (still only while the guard allows it).

When to change it: almost never. `always` is louder in theory only, harder on the earpiece, and costs idle power. Re-read every cycle.

7.6 WATCH_INTERVAL

How often, in seconds, the daemon re-asserts the routing. Default `2`.

The audio HAL resets mixer controls on every route change, so this cannot be very large. `2` is a good balance; raise it to `5` if you notice battery drain.

> **TIP**
> A longer interval also means the earpiece joins in a little later after you press play, the amp's right slot comes a little later at the start of a stream, and the crash-loop guard looks less often.

7.7 STARTUP_DELAY

Seconds to wait after boot before touching anything. Default 10. It gives the audio HAL time to settle after Android has started.

When to change it: normally never. `stereoctl restart` ignores it and starts at once.

7.8 REQUIRE_SPEAKER_ROUTE

Skip the earpiece when headphones or Bluetooth audio are connected, by checking Android's audio state. Default 0 (off).

It costs one `dumpsys` call per cycle, so it is off by default: the routing usually falls away on its own when the output changes, and on the CMF Phone 1 the guard (next section) already does this job with a single mixer read.

When to change it: on another device without a usable guard, if the earpiece keeps playing with headphones in.

> TECHNICAL
>
> Set to 1, it reads the first 200 lines of `dumpsys audio` and skips the earpiece if they mention `BLUETOOTH_A2DP`, `WIRED_HEADPHONE`, `WIRED_HEADSET` or `USB_HEADSET`. If the dump is empty, it does not block.

7.9 GUARD_CTL and GUARD_VALUE

Apply the routing only while the mixer control GUARD_CTL reads GUARD_VALUE. Leave GUARD_CTL empty to switch the check off.

On the CMF Phone 1 the installer sets:

```
GUARD_CTL='aw_dev_0_switch'
GUARD_VALUE='Enable'
```

That is the speaker amp's switch, which the HAL turns off when headphones or Bluetooth take over. So the earpiece follows the speaker rather than playing to nobody.

When to change it: not on the CMF Phone 1 (the installer sets it again on every install). Use stereoctl guard off or solo for a temporary bypass.

> TECHNICAL
>
> One mixer read per cycle, no dumpsys. If the control cannot be read at all, the guard does not block.

7.10 PLAYBACK_CTL and PLAYBACK_VALUE

How the module tells that audio is playing. PLAYBACK_CTL names one or more mixer controls, separated by |; audio is playing when any one of them reads PLAYBACK_VALUE (default 1).

On the CMF Phone 1 the installer sets all six DSP playback task switches:

```
PLAYBACK_CTL='dsp_music_runtime_en|dsp_fast_runtime_en|dsp_primary_runtime_en|dsp_deepbu
f_runtime_en|dsp_voipd|_runtime_en|dsp_offload_runtime_en'
PLAYBACK_VALUE='1'
```

Empty means: scan /proc/asound for a running stream. That scan misses DSP playback entirely, so on devices where media bypasses the normal path, name a control the HAL flips when media starts. Find one with stereoctl diff.

When to change it: not on the CMF Phone 1 (set again on every install).

> TECHNICAL
>
> Measured on the phone, logging the switches each second: a game turns on dsp_fast_runtime_en; a music player dsp_music_runtime_en with dsp_primary_runtime_en (dsp_deepbuf_runtime_en for a moment); nothing stays on when playback stops. dsp_playback_runtime_en stays on even when idle, so it is not used. Watching dsp_music alone (up to v1.0.2) left games on the speaker only. If none of the named controls can be read (renamed by a vendor update, say), the module falls back to the procfs scan rather than reporting "never playing" for ever.

7.11 MIXER_CARD

The ALSA sound card to talk to. Empty (default) means tinymix's default, card 0, which is right on essentially every phone.

When to change it: only if a probe shows several sound cards and the routing is on another one.

7.12 CHANNELS

Value	Meaning
`mono` (default)	Stock. Both drivers play the full mix; Awinic speaker protection.
`stereo`	Left and right on the earpiece and the speaker, following the screen. Awinic's DSP speaker protection is off for media (calls keep it); the attenuation and the module's protection stand in.

Change it with `stereoctl channels mono|stereo`, the apps or the Action button, not by hand: Stereo needs its files staged. The module itself sets it back to `mono` when a safety net removes Stereo.

> TIP
>
> The comment above this setting in `stereo.conf` still says that both modes need a reboot and that the earpiece plays the left channel and the speaker the right. Both are out of date: switching has been live since v0.9.4, and left and right follow the screen since v0.9.5.

7.13 STEREO_ATTEN_DB

In Stereo, hold the speaker amp this many dB down. Range 0–30, default 6. This is the apps' speaker level: 10 = 6 dB, 2 dB more per level down (section 5.3).

Together with the protection, it is part of the margin that stands in for Awinic's protection, on top of the amp's own over-temperature and over-current cut-outs.

When to change it: raise it if you play loud, or to balance the speaker against the earpiece.

> WARNING
>
> Values under 6 dB give up part of the safety margin the module was designed with. The clean app never goes there.

7.14 The protection settings

Setting	Values	Default	What it does
SPEAKER_BOOST	0–6	0	dB less amp attenuation, only while the protection runs; the speaker's peaks still never exceed what Awinic allows in Mono (−5 dB).
BASS_PROTECT	off, light, medium, strong	medium	Holds the speaker's band below 100 Hz at −8 / −14 / −20 dB below the amp's full output, only when louder.

EARPIECE_LIMIT	on, off	on	Holds the earpiece at −2.5 dBFS (it clips above).
EARPIECE_LEVELLER	off, light, medium, strong	off	Raises the earpiece's quiet and medium passages by at most 4 / 8 / 12 dB; no frequency changed, peaks never above today's, near-silence left alone.

Change them with `stereoctl protect` or the apps. Chapter 5 explains each one and when to change it.

7.15 LOG_LEVEL and LOG_MAX_KB

Setting	Values	Default
LOG_LEVEL	`debug` (everything), `info` (normal events), `warn` (problems only)	`info`
LOG_MAX_KB	size in kilobytes before the log is rotated	512

When to change it: set `debug` while chasing a problem, `warn` to keep the log small.

7.16 actions.conf: the routing

`actions.conf` holds the routing, one action per line, applied from top to bottom. Blank lines and lines starting with `#` are ignored.

```
ctl|<mixer control name>|<value>    set an ALSA mixer control
sysfs|<path>|<value>               write to a sysfs node
prop|<property>|<value>             set a system property
```

Before changing anything, the module saves the original value of everything it touches under `/data/adb/cmf-stereo/state/`, and restores it when playback stops, the module is turned off, or it is uninstalled.

> WARNING
>
> On another device this file ships **empty by design**: the control names must come from a real probe. A wrong write is at best silence and at worst a damaged earpiece. When you first find a working route, set its gain low and raise it one step at a time. Distortion or a rattle means back off at once.

> TECHNICAL
>
> A `ctl` line whose control does not exist is skipped with a warning in the log. A `sysfs` write denied by SELinux is logged. A `prop` is set with `resetprop` where available, otherwise `setprop`. An unknown kind is logged and skipped.

7.17 The CMF Phone 1 routing profile

On a CMF Phone 1, `actions.conf` holds three lines, verified on Nothing OS B4.1-260615-1652, Android 16:

```
ctl|Handset Volume|0
ctl|ADDA_DL_CH1 DL_24CH_CH1|1
ctl|RCV Mux|Voice Playback
```

1. **Handset Volume 0**: the earpiece gain, +8 dB, set before it is driven.
2. **ADDA_DL_CH1 DL_24CH_CH1**: feed the internal DAC from the same stream that already goes to the speaker.
3. **RCV Mux = Voice Playback**: route that DAC to the earpiece.

Order matters: the feed is connected before the mux is opened, so the receiver is never pointed at an undefined path. These three lines are the same in Mono and Stereo; the earpiece always takes channel 1 of the stream, the left channel.

> TIP
>
> To make the earpiece quieter, raise the first number towards 18, one step at a time. Back off at the first buzz or rattle.

08

SAFETY AND RECOVERY

The module is built so that a mistake costs a reboot, not a trip to recovery. This chapter describes each safety net, what triggers it, and how to recover by hand if you ever need to.

8.1 The safety nets at a glance

Safety net	Protects against	What it does
Boot watchdog	a boot that fails with the module's overlay in place	after two failed boots, removes the overlay; Stereo goes back to Mono
Crash-loop guard	the audio service dying again and again in Stereo	3 restarts within a minute: back to the stock files (Mono) at once
Hash checks	patching or mounting the wrong files	only the exact analysed build is patched; every result is checked
Unknown-build guard	a Nothing OS update	stock Mono until a module update supports the new build
Live switch checks	a switch going wrong	every file read back; audio service must stay up; otherwise stock files
Ships empty elsewhere	wrong mixer writes on another phone	changes nothing until real routing is filled in
XML validation	a broken audio policy	a patched policy is installed only if it passes a structural check
Restore on stop	leftover changes	every touched control put back on stop, off or uninstall

The boot watchdog, the crash-loop guard and a failed live switch ending on the stock files are covered by tests on fake files. None has been needed on the phone yet.

8.2 Boot watchdog

If two boots in a row fail to finish while the Stereo overlay (or the audio policy overlay) is in place, the module removes that overlay before it can be mounted again. Stereo goes back to stock Mono (`CHANNELS=mono`). A bad boot costs one extra reboot instead of a trip to recovery.

After the next good boot, the log notes it: `the stereo overlay was auto-removed after failed boots; channels are back to mono`.

> TECHNICAL
>
> Each boot with something active (an overlay, or any routing action) adds a strike to `/data/adb/cmf-stereo/boot_pending` in `post-fs-data.sh`. `service.sh` clears it once `sys.boot_completed` is 1 (it waits up to 180 s). At two strikes, `post-fs-data.sh` removes the overlay files before Magisk mounts them, sets `CHANNELS=mono` and leaves a marker (`stereo_auto_removed` or `xml_overlay_auto_removed`) that `service.sh` logs and deletes. `stereoctl status` shows the count on its `boot:` line.

8.3 Crash-loop guard

In Stereo the audio service runs the module's own code. If it kept crashing, Android would simply restart it and the phone would still boot, so the boot watchdog would never notice. The crash-loop guard covers that case: if the audio HAL restarts 3 times within a minute in Stereo, the daemon puts the stock files back (Mono) at once and tells the apps.

The log then says: the audio HAL restarted 3 times within a minute in stereo - back to mono (stock files). You can switch back to Stereo yourself once you know why.

> TECHNICAL
>
> The daemon records the HAL's process id (`init.svc_debug_pid.vendor.audio-hal`) each cycle in `state/hal_pids` and keeps only the last 60 s; four different ids (the first seen plus three restarts) trigger it. A mode switch restarts the HAL once, which counts as one. If the live switch to Mono fails, the log says Mono comes after a reboot.

8.4 Hash checks on every file

The module never patches or mounts an audio file it has not identified exactly. Each file's SHA-256 fingerprint must match the analysed build before it is patched, and the patched result must match the tested one before it is used.

- A different file — another firmware, another region's build, a file changed by another module — is refused, and the phone stays on Mono.
- Files from older Stereo versions (v0.9.0 to v1.0.3) are recognised and converted to the current one.
- The module's own protection library is checked against its own hash. Without a matching library, Stereo still plays (the patch falls back to the rotation swap alone) and the speaker keeps its fixed attenuation.

> TECHNICAL
>
> The expected hashes are in scripts/channels.sh (section 12.6). hal-swap/run.sh checks the bytes channels.sh writes against the assembled patch, in an ARM64 emulator.

8.5 The unknown-build guard

At every boot, before anything is mounted, the module checks the phone's own vendor audio files against the analysed build. If they differ (after a Nothing OS update, say), the Stereo overlay is removed and the phone boots on stock Mono. Mounting the old patched files over a new build would run an old audio HAL against new firmware and configs.

The card then says [Stereo unavailable] and stereoctl status shows build: updated, until a module release supports the new build. The log says: this Nothing OS build is not supported yet - stereo is unavailable, mono runs.

8.6 Ships empty on other devices

Mixer control names differ from device to device. A wrong write is, in ascending order of regret: nothing, silence, a crash of the audio HAL, or an earpiece driven past what a receiver is built for. The earpiece has a fraction of the speaker's movement and power handling, and no protection circuit that assumes you will use it for music.

So on any phone that is not a CMF Phone 1, the module installs with an empty actions.conf and changes nothing. The daemon starts, finds no actions, logs no actions

configured - nothing to do (run 'stereoctl probe' first) and exits. Probe first, then fill in real names.

> WARNING
>
> When you find a working route on another device, start its gain low and raise it a step at a time. Distortion or rattle means back off — that damage does not come back.

8.7 XML validation

No patched audio policy is installed unless it passes a structural check. Android has no xmllint, and a malformed audio policy takes audioserver down hard enough to loop the boot.

> TECHNICAL
>
> Checked: balanced <module>, <modules>, <mixPort>, <mixPorts>, <devicePort>, <devicePorts>, <route>, <routes> and root elements (self-closing ones count as open and close), the closing root tag present, and no stray trailing backslash from a bad insertion. This applies to stereoctl xml-patch, which the CMF Phone 1 does not need.

8.8 Everything put back

Before the module changes a control, it saves its original value once. It puts that value back:

- when playback stops (in `MODE=playback`);
- when you turn the module off;
- when the daemon stops;
- when you uninstall the module.

The speaker amp's own gain is given back when you go to Mono or uninstall.

> TECHNICAL
>
> Saved originals are in `/data/adb/cmf-stereo/state/`, one file per control. They survive a reinstall; `stereoctl reset` clears them if one is ever wrong.

8.9 Manual recovery

If you ever need to switch the module off without booting normally:

```
adb wait-for-device shell         # or hold vol-down for safe mode
su
touch /data/adb/modules/cmf_stereo/disable
reboot
```

Both Magisk and KernelSU honour the `disable` file, and the boot script never touches audio when the actions list is empty.

> TIP
>
> Thanks to the boot watchdog you should rarely need this: two failed boots with Stereo on already put the phone back on Mono by themselves.

8.10 Reporting a security problem

The module runs as root. If you find a security problem, **do not open a public issue.** Report it privately on the project's GitHub page: open the **Security** tab and choose **Report a vulnerability.** Only the maintainer sees it.

In scope: anything that lets another app, file or user run root code through the module's scripts (including via its config or state files); the module writing, deleting or mounting outside what it documents; staging accepting or producing a vendor file other than the verified ones, or the hash checks being bypassable; a bootloop the watchdog does not recover from; anything that can drive the speaker or earpiece past safe levels without you asking for it.

Include the version, your phone model and build, what happens and how, and logs. Only the latest release gets fixes.

09

TROUBLESHOOTING

Start with the first section: three quick looks settle most problems. Then find your symptom below.

9.1 First steps for any problem

1. Look at the module's card. Its first words say what runs (section 3.2).
2. Run `stereoctl status` with music playing (section 6.3).
3. Read the log: `stereoctl log 100`. Lines marked `[warn]` usually name the problem.
4. Make sure you are on the latest release. Only the latest gets fixes.
5. Think about other modules that change audio (section 9.9).

9.2 The earpiece is silent

Start music, then run:

```
stereoctl doctor
```

It walks every layer that can silence the earpiece and names the one responsible, with its fix. Fix the items it marks, then run it again. The next section explains each message.

Also check:

- headphones or Bluetooth are not connected (the earpiece steps aside for them);
- the media volume is up;
- `stereoctl solo`, to hear the earpiece alone, then `stereoctl unsolo`.

9.3 Reading doctor's messages

Step	Message	Fix
1. module armed	NO - kill switch set (it survives reboots).	stereoctl on
2. daemon	NOT running.	stereoctl restart
2. daemon	N RUNNING - they fight over the mixer.	stereoctl restart
3. tinymix	UNUSABLE - mixer writes cannot happen	see section 9.13
4. playback	IDLE - with MODE=playback nothing is applied until audio plays.	start music and run doctor again
5. guard	BLOCKING: aw_dev_0_switch = ..., needs Enable	the speaker amp is off (headphones, or turned off by hand): stereoctl solo, or re-enable the amp
6. applying now	FAILED <control> want X, reads Y	the control did not take its value; check the name in actions.conf

If everything applied and it is still silent, doctor says: Everything applied. If it is still silent, the path is powered but inaudible - check Handset Volume and the media volume.

> TECHNICAL
>
> Step 4 also prints which detector answered: `mixer controls: …` (the DSP task switches) or `procfs substream scan` (the fallback, which misses DSP playback). Step 5 says `bypassed (solo mode)` or `none configured` when it does not apply.

9.4 No stereo: both drivers play the same

1. Check what runs: `stereoctl channels`. If it says `running: mono`, switch: `stereoctl channels stereo`.
2. If it says `staged: stereo (reboot to apply)`, reboot.
3. If the card says `[Stereo unavailable]` or `status` shows `build: updated`, your build is not supported yet (section 4.11).
4. If Stereo was on and is now Mono by itself, a safety net switched it (section 9.12).
5. Test with `LEFT_ONLY_3min.wav`: in Stereo, earpiece on your left, only the earpiece should play.
6. Many recordings have little difference between left and right. The test files show clearly whether Stereo works.

9.5 Stereo refuses to start

If `stereoctl channels stereo` prints `stereo NOT staged:` followed by a reason, your phone's files could not be prepared. Your phone stays on Mono, unchanged.

Reason	Meaning
`… missing - not a CMF Phone 1 audio stack`	The audio files the module patches are not there.
`… is not the analysed build (<hash>) - refusing to patch an unknown binary`	Your audio HAL differs from the analysed one: another Nothing OS build or region, or a file changed by another module.
`/vendor/etc/aurisys_config_rv.xml is not the analysed file - refusing`	The same, for the DSP configuration file.
`patched … does not match the tested binary` / `… does not match the expected result`	The patch did not produce the tested file; nothing is used.
`cannot copy …` / `patching … failed`	A copy or write failed (storage full, permissions).

If you are on a build other than the tested one (section 1.4), Stereo has to wait until that build is checked and added in a module release.

9.6 Left and right are reversed

- Make sure **auto-rotate is on**; left and right follow the screen's rotation.
- For apps with their own audio effect (VLC), wait about **2 seconds** after pressing play or turning the phone.
- If two apps play at once, one of them with its own effect, one can come out reversed with the earpiece on your right. This is a known limit (section 4.12).
- Check with `LEFT10s_RIGHT10s.wav`: the voice says which side should play.

9.7 Distortion, crackle or rattle

> WARNING
>
> **Back off at once.** Damage to a driver does not come back.

From the speaker (in Stereo):

- lower the speaker level (raise `STEREO_ATTEN_DB`);
- reduce or remove the speaker boost;
- make sure the deep bass guard is on; try `strong`;
- for loud listening, switch to **Mono**.

From the earpiece:

- in Stereo, make sure the earpiece limiter is on (`stereoctl protect earpiece on`);
- make the earpiece quieter: raise the `Handset Volume` number in `actions.conf` (or lower the app's earpiece level);
- remember that in Mono the earpiece limiter does not run.

> TECHNICAL
>
> Measured in Stereo without Awinic: the speaker's distortion grows steadily with level from about −12 dBFS, mostly odd harmonics, to about 27 % at −3 dBFS at 1.2–2 kHz; the earpiece stays under about 5–10 % up to about −3 dBFS from 150 Hz up, and clips in its top 1.5 dB.

9.8 The sound stops, drops out or pauses

- **Right after a switch between Mono and Stereo:** this is expected. The audio service restarted; press play again.
- **Stereo turned into Mono by itself:** the crash-loop guard found the audio service restarting again and again. Check `stereoctl log` (section 9.12).
- **The earpiece comes and goes with `MODE=playback`:** that is the module following playback. If it misses some apps, make sure you are on v1.0.3 or newer (games were not detected before).
- **The earpiece joins a moment after you press play:** up to one cycle (2 s by default).
- **Another module** may be fighting over audio (next section).

9.9 Full volume not reached: check other modules

Other modules that change audio can interfere with this one or with the phone's volume. **A real case from the test phone:** NothingTweaks' Customise Volume Steps, set to 24 steps, made the highest volume step silent, and the middle of the bar was the loudest; it looked like this module's fault even with the module off. With the custom steps turned off, full volume was correct again.

If something sounds wrong that this manual does not explain:

1. list your modules, especially any that touch audio, volume, Dirac or effects;
2. disable them one at a time and reboot after each;
3. test again with the test files and full media volume.

> WARNING
>
> Do not run the Hi-Res Audio module at the same time: the two touch the same files. A module that mounts over the same audio files also makes a live switch refuse with a mount over the audio files is not ours - not touching it.

9.10 The earpiece is weak or uneven

- Check Handset Volume in actions.conf: 0 is the loudest (+8 dB). 31 is the codec's mute value, not the top. The installer corrects old values above 18, but check if you edited the file by hand.
- If the log keeps showing Handset Volume: 0 -> 31, the file has the old value: the daemon pulls the earpiece down every cycle while the HAL puts it back. Set it to 0.
- The earpiece will never match the speaker. Balance by turning the speaker down, or try the earpiece leveller (Stereo).

9.11 The card says Not running, or the daemon misbehaves

If the card ends with Not running: restart it in the app., or status says daemon: not running:

```
stereoctl restart
```

If it does not start, read stereoctl log. Possible reasons:

- no actions configured - nothing to do (run 'stereoctl probe' first): actions.conf is empty (normal on other phones);
- another daemon is already running (pid …) - exiting: one is already running; that is fine;
- tinymix not found but ctl actions are configured; they will be skipped: see section 9.13.

If doctor reports several daemons, stereoctl restart stops them all and starts one.

9.12 Back in Mono by itself

If you chose Stereo and find Mono running, one of the safety nets switched it. The log tells you which:

Log line	Cause
boot watchdog: 2 failed boots with the stereo overlay - back to mono	Two boots failed with Stereo on.
the audio HAL restarted 3 times within a minute in stereo - back to mono (stock files)	The audio service crash-looped in Stereo.
the vendor audio files are not the analysed build (…; a Nothing OS update?) - stereo overlay removed, mono	Your audio files changed.

In the first two cases you can try stereoctl channels stereo again, but if it happens again, stay on Mono and report it with your log.

9.13 tinymix problems

The module needs `tinymix` to read and write mixer controls. The CMF Phone 1 ships none, so the module bundles one. If `status` says `NOT FOUND` or `UNUSABLE (wrong arch, or not executable)`:

- reinstall the module, so its bundled `bin/tinymix` gets its permissions again;
- or build tinymix yourself, or use Termux's `tinyalsa` package, and place it at `/data/adb/cmf-stereo/bin/tinymix` with `chmod 0755`.

Section 12.7 describes the bundled binary and the search order.

9.14 What to send when asking for help

- the module version, your phone model and the Nothing OS build (compare it with the tested build in section 1.4);
- the output of `stereoctl status` and `stereoctl doctor`, with music playing;
- the output of `stereoctl channels` and `stereoctl protect`;
- `stereoctl log 200`;
- the other modules you have installed;
- on another device: `stereoctl probe`, then `stereoctl report`, and send `/sdcard/cmf-stereo-report.txt`.

10

QUESTIONS AND ANSWERS

Short answers to the questions people ask most, with a pointer to the full explanation.

10.1 Compatibility

Does it work on other phones? It works on the CMF Phone 1 (A015). On any other phone it installs and does nothing; you can probe the hardware there (section 2.9).

Does it work on Global or India builds? Untested. The module checks the audio files by hash; if they differ from the tested EEA build, it refuses Stereo and stays in Mono.

Does it need a custom kernel? No. The routing, the amp driver, the audio HAL and the DSP libraries are all vendor code; root is enough.

Magisk, KernelSU or APatch? All three. The Action button needs Magisk 28 or newer.

Can I use it with the Hi-Res Audio module? No, not at the same time: they touch the same files.

10.2 Sound

Will the speaker get louder? No. The main speaker's own path is untouched; in Stereo it is held a little lower for safety. Mono is louder overall because both drivers play.

Why does the earpiece sound quieter than the speaker? It is a receiver, made for voice. It already plays at its maximum. Turn the speaker down to balance, or try the leveller.

Does it change Dirac, Nothing's sound tuning? No. Dirac keeps running on the output mix in both modes.

Does hi-res playback still work? Yes, checked in Stereo with USB Audio Player Pro (Direct 192 kHz).

Are headphones affected? No. The earpiece steps aside, and the rotation swap applies only to the speakers.

10.3 Safety

Can Stereo damage my speaker? Stereo turns off Awinic's speaker protection for media. The module's protection stands in with fixed limits measured against Awinic's, but without a temperature model. For long, loud listening use Mono; in Stereo lower the speaker level and back off at the first distortion.

Can it damage my earpiece? The earpiece has no protection circuit. In Stereo the earpiece limiter keeps it below where it crackles; in Mono it plays without it. Back off at the first buzz.

Does it write to the vendor partition? Never. Its files are mounted over the vendor paths by your root manager, and removed with the module.

What if the phone does not boot? After two failed boots the module removes Stereo by itself. Manual recovery is in section 8.9.

Are calls affected? No. Calls keep Awinic's protection, and a mode switch waits during a call.

10.4 Everyday questions

Do I need to reboot to switch modes? No, since v0.9.4. The sound stops for about a second; press play again.

Do I need the app? No. The module works on its own; the app, the Action button and `stereoctl` are three ways to control it.

Does it drain the battery? Very little: one check every 2 seconds. Raise `WATCH_INTERVAL` to 5 if you notice drain.

Why does the first second of a song sometimes come from one side? The amp driver resets its slot at every stream start; the module corrects it on its next cycle (section 4.9).

What happens after a Nothing OS update? The phone boots on stock Mono, the card says Stereo unavailable, until a module update supports the new build.

Are my settings kept when I update? Yes. They live in `/data/adb/cmf-stereo/`, which updates do not touch.



HOW IT WORKS

First the idea in plain words, then each mechanism in technical detail: the routing, the two places the phone folds to mono, the daemon, the rotation swap, playback detection, the live switch and the boot scripts.

11.1 In plain words

Your phone has two drivers: the bottom speaker for media, and the earpiece for calls. Normally media never reaches the earpiece at all.

The module does three things:

1. **It connects the earpiece** to the same sound the speaker plays, by setting three switches in the phone's audio mixer. That alone gives you Mono: both drivers play everything.
2. **For Stereo, it stops the phone mixing left and right together.** Two places in the phone's audio software do that, because the phone was designed with one speaker. The module switches both off, and the earpiece and the speaker each get their own channel.
3. **It protects the drivers** in Stereo, since one of those two places was also Awinic's speaker protection.

A small background service keeps those mixer switches set while you listen, because the phone's audio software resets them every time a sound starts.

11.2 Three layers, smallest hammer first

1. **Mixer actions (`actions.conf`).** The real work. An ALSA mixer control enables the receiver's output stage; on MediaTek there is usually also a routing control that decides which DL (downlink) stream feeds it. Setting those while the speaker plays puts the earpiece to work next to the speaker.

2. **Audio policy overlay (`stereoctl xml-patch`, optional).** If the policy declares the speaker as `AUDIO_CHANNEL_OUT_MONO`, the framework downmixes to one channel before the HAL sees it, and no mixer work brings the second channel back. The patch widens only the speaker `<devicePort>` to stereo, in an overlay copy. Opt-in, validated, reversible — and not needed on the CMF Phone 1.

3. **What the CMF Phone 1 actually needed:** the internal DAC fed from the speaker's stream (next sections), and, for Stereo, the two mono folds switched off.

11.3 How the earpiece was brought in

Media on this phone never touches the internal codec. Playback runs:

```
DL_24CH_CH1/CH2 --> I2SOUT4_CH1/CH2 --> Awinic aw_dev_0 smart PA --> speaker
                          <-- I2SIN4 --> UL3   (IV-sense feedback)
```

The earpiece hangs off the internal codec's DAC (`ADDA_DL`), and that DAC normally has no source connected at all: every one of its roughly 20 input switches sits Off. So pointing `RCV Mux` at the receiver routes it to a silent wire — exactly what happened on the first attempt.

The switch that matters is `ADDA_DL_CH1 DL_24CH_CH1`: it feeds the internal DAC from the same `DL_24CH` stream already going to the speaker amp. Turn it on, then open `RCV Mux`, and the earpiece plays.

There is only one Awinic device (`aw_dev_0`), so there is no second amp channel to press into service; the internal codec was the only way in. That alone gives both

drivers the same full mix: Mono.

11.4 The two places the phone folds to mono

Real Stereo needed one more step: switching off the two places the phone folds the speaker to mono.

1. **The MediaTek audio HAL.** `AudioALSAPlaybackHandlerBase::doStereoToMonoConversionIfNeed` writes (L+R)/2 to both channels whenever the phone is configured for one speaker. Patching it out alone: still mono.
2. **Awinic's speaker protection on the audio DSP** (Aurisys `smartpa_awinic`). It has one speaker model, so it mixes L+R and protects that. With it replaced by MediaTek's pass-through `aurisys_demo` as well, the amp's right slot went silent with a left-only file, and the left slot still played: the channels were separate at last.

Earlier versions of the README said both drivers were hard-wired to channel 1, then that a custom kernel would be needed, then that the DSP firmware was the limit. All three were wrong; each stage was then measured (section 12.1).

> TECHNICAL
>
> Setting `MTK_AUDIO_NUMBER_OF_SPEAKER` to 2 was rejected: it also switches the HAL to a `dual_speaker_output` path that does not exist in this phone's `audio_device.xml`, and puts Awinic's protection into two-amp mode with one amp fitted.

11.5 From the app to the two drivers

The path of a sound in Stereo:

```
app -> Android's mixer (AudioFlinger)          swaps L/R itself at rotation 270,
                                            except a track with its own effect
    -> Dirac (Nothing's tuning, session 0)
    -> audio HAL, AudioALSAPlaybackHandlerBase   patched: hal-swap/patch.S loads
       ::doStereoToMonoConversionIfNeed         protect/libcmfstereo.so and calls it
         rotation swap (VLC-type outputs)       <- vendor.audiohal.cmf_swap (daemon)
         speaker: bass guard, peak limiter      <- vendor.audiohal.cmf_*_ceil (daemon)
         earpiece: leveller, limiter
    -> audio DSP, aurisys_demo (pass-through)    aurisys_config_rv.xml patched
    -> DL_24CH ch1 -> codec DAC -> earpiece      routing: config/actions.cmf1.conf
       DL_24CH ch2 -> Awinic amp (right slot) -> speaker   amp slot/attenuation: daemon
```

Measured end to end with the HAL's and the DSP's own recordings: nothing after the patched function changes the level, so the library's ceilings are exact.

11.6 The protection library

The module's protection is a small library of its own, libcmfstereo.so, which the patched audio HAL loads once and calls for every buffer it plays, just before the DSP.

TECHNICAL

For a speaker-only stream (devices exactly AUDIO_DEVICE_OUT_SPEAKER), 16- or 32-bit, two interleaved channels, it runs:

1. the rotation swap: left and right swapped at rotation 270 for the outputs vendor.audiohal.cmf_swap names;
2. speaker (right): the bass limiter (band below 100 Hz, only above its ceiling), then a peak limiter at its ceiling;
3. earpiece (left): the leveller if on, then a peak limiter at its ceiling.

Ceilings come from properties in hundredths of a dB of full scale; a ceiling at or above 0 dBFS, or unset, switches that stage off and leaves the samples exactly as they were. It never blocks, never allocates, and touches only the buffer it is given. If the library cannot be loaded, the patch does not retry and runs the rotation swap alone. Source: protect/cmfstereo.c, with host tests and a reproducible NDK build (protect/build.sh) that rebuilds byte for byte and needs only libc, libm and libdl.

11.7 The background daemon

The audio HAL rewrites mixer controls on every route change (a stream starting, a device switch, a call), so a one-shot write does not survive. A small daemon re-asserts the values every couple of seconds and, by default, only while audio is playing.

TECHNICAL

Each cycle (`WATCH_INTERVAL`, 2 s), `scripts/stereo-daemon.sh`:

1. re-reads `stereo.conf`, so edits take effect without a reboot;
2. if disabled, reverts once and waits (in Stereo, keeps the amp on its mono slot while playing);
3. decides whether the routing is wanted: playing (or `MODE=always`), speaker route allowed, guard satisfied;
4. applies the actions (a no-op for controls already right), or reverts them when playback stops;
5. in Stereo: checks the crash-loop guard; hands the library its ceilings; holds the amp attenuation (when playback starts, every 10th cycle, or when the wanted value changes); at rotation 270 while playing, updates the outputs to swap (when playback starts, when the phone turns, and every 3rd cycle); sets the amp to its right slot while the earpiece plays, or its mono slot while playing without it.

Only one daemon runs: it takes an atomic lock at start, and others exit. On stop it reverts the actions, puts the amp on its mono slot in Stereo, and updates the card.

11.8 How turning the phone is handled

Android itself swaps left and right before the audio HAL when the phone is turned with the earpiece on your right (rotation 270). The module lets it, and only fills the one gap Android leaves.

TECHNICAL

Measured with the HAL's own recordings (`vendor.streamout.pcm.dump`): a left-only beep reached the HAL on the right at rotation 270 and on the left at 90, in all four Dirac tunings and from USB Audio Player Pro — except for a track whose session has its own effect chain (VLC's Dynamics Processing, the test app's MOVIE + EFFECT), which reached the HAL unswapped.

The daemon reads `dumpsys media.audio_flinger` only while playing at rotation 270, finds the tracks with their own effect chain (session 0, the output mix with Dirac, is not counted), and writes the OR of their mixer threads' output flags to `vendor.audiohal.cmf_swap`. The patched HAL keeps Android's `rotation` parameter in `vendor.audiohal.cmf_rotation` (the stock HAL drops it) and swaps a speaker stream only when the rotation is 270 and its output is named.

v0.9.5 to v1.0.0 swapped by output, by Dirac tuning, then by patching Android's mixer; each rule held only for the apps it was tested with. v1.0.1 returned to letting Android swap, and v1.0.2 added the swap for own-effect tracks.

11.9 How playback is detected

To use the earpiece only while something plays, the module has to know when that is. On this phone the obvious way (looking for a running stream in `/proc/asound`) never sees media, because media plays through the audio DSP.

> TECHNICAL
>
> So the module reads the DSP's own task switches, dsp_<task>_runtime_en, for every playback task: music, fast, primary, deep buffer, VoIP and offload. Audio counts as playing when any of them reads 1. If none can be read, it falls back to the procfs scan. Before v1.0.3 only the music task was watched, so games (which use the fast task) played on the speaker only.

11.10 How a live switch works

Instead of rebooting, the module restarts only the audio HAL on the wanted files. Measured on the CMF Phone 1: the HAL comes back within seconds, audioserver reconnects on its own, calls keep working, and players pause.

TECHNICAL

1. If the files the system shows already match the wanted mode: `unchanged`, nothing done.
2. Refuse during a call.
3. For Stereo, check every staged file's hash before touching anything.
4. Take off the module's own mounts on the three paths (lazily, topmost first); refuse if any mount that is not the module's remains.
5. Mount what the mode needs (via `nsenter` into init's mount namespace when run from an app), and read each file back through `/proc/1/root`.
6. `setprop ctl.restart vendor.audio-hal`; wait for a new process (up to 8 s), then make sure it is still the same process 3 s later.
7. On any failure: undo, mount the checked stock copies over whatever is not stock, and restart the HAL only if all three files read back as stock.

Why the care: Magisk 31 builds `/vendor/lib64/hw` as a tmpfs skeleton whose replaced files are empty placeholders under the module's mounts. In v0.9.8, restarting the HAL on those empty files had it and `audioserver` crash-looping until a reboot (fixed in v0.9.9).

11.11 What the boot and button scripts do

Script	When it runs	What it does
`customize.sh`	install and update	device check, config kept across updates, Stereo re-staged (section 2.5)

post-fs-data.sh	early boot, before mounts	boot watchdog for both overlays; unknown-build guard; clears the old daemon lock; counts a boot strike if anything is active
service.sh	late boot	waits for boot to complete, clears the strike, logs auto-removals, saves the hash-checked stock copies, notes the build state, updates the card, starts the daemon
action.sh	the Action button	Mono ⇔ Stereo through stereoctl channels
uninstall.sh	module removed	stops the daemon, restores every touched control, gives the amp its gain back; keeps /data/adb/cmf-stereo/

12

TECHNICAL REFERENCE

The measurements behind the module, every control, property and file it uses, the bundled tinymix, the probe's output, the development checks and the version history.

12.1 Findings: where the speaker is folded to mono

Each stage was measured with a test file that has sound on the left channel only and silence on the right, with the amp switched to its right slot: silence there means the right channel really is separate.

Stage	Separate L/R?	Evidence
Android framework	Yes	`dumpsys media.audio_flinger`: speaker output is 2-channel FL/FR, `Master mono` off. The policy's speaker port is already stereo, so `stereoctl xml-patch` finds nothing to change.
Dirac (Nothing's speaker tuning)	Not the fold	Session-0 insert effect, 2-in/2-out presets. Removing it alone changed nothing.
MediaTek audio HAL (`audio.primary.mt6878.so`)	Folds to mono	`doStereoToMonoConversionIfNeed` writes (L+R)/2 to both channels whenever `MTK_AUDIO_NUMBER_OF_SPEAKER` is 1. Patching it out alone: still mono.
Aurisys on the audio DSP, `smartpa_awinic`	Folds to mono	One speaker model (`AW_DSP.bin`: five presets, each one 11,244-byte block for one speaker, all with 0.5/0.5 weights). With `aurisys_demo` as well, the right slot went silent and the left still played.
AFE routing	Yes	`I2SOUT4_CH2 <- DL_24CH_CH2` is live during playback.
Awinic amp (`aw882xx`)	Yes	I2S slot select is register `0x06` bits 11:10 (`01` left, `10` right, `11` mono (L+R)/2), writable through the driver's `reg` attribute. Stock is left (`0x34e8`); on the right slot, cutting `I2SOUT4_CH2` silences the speaker.

Earpiece	Left channel	RCV <- ADDA_DL_CH1 <- DL_24CH_CH1; ADDA_DL_CH1 has no CH2 source.

12.2 Findings: protection and the drivers

Measured with CMF L/R Test DRIVER STUDY (tones 100 Hz to 8 kHz at −30 to −3 dBFS; the HAL's and the DSP's recordings, and the HAL's raw 24-bit two-microphone capture). The Awinic-against-module table is in section 5.11. Also measured:

- the speaker's resonance at about 1 kHz (from Awinic's current and voltage sense);
- Dirac (MUSIC) before either driver: about +2 dB at 150 Hz, +5 to +6 dB at 200–500 Hz, −2 to −4 dB at 3–5 kHz, a limiter at about −0.2 dBFS, and a second harmonic at −16 dB on 100 Hz (bass enhancement); above 150 Hz what reaches the drivers is clean (under 0.1 % distortion);
- the earpiece's whole path is at full level already: the beep was −8.7 dBFS from Android's output to the HAL and through the DSP, then ADDA_DL_GAIN −0.3 dB and the receiver amp at its top step (+8 dB);
- earpiece distortion under about 5–10 % up to about −3 dBFS from 150 Hz up, clipping in its top 1.5 dB;
- speaker (Awinic off) distortion growing from about −12 dBFS to about 27 % at −3 dBFS at 1.2–2 kHz;
- the limiters add at most 1.2 % distortion while working (50 Hz), under 0.1 % above 1 kHz.

Still pending: the Mono part of the study (Awinic on) for the same tones.

12.3 Mixer controls and hardware

Control	Role	Module's use
Handset Volume	receiver amp gain, MT6369 codec (0 = +8 dB … 18 = −10 dB, 31 = mute)	set to 0 while playing
ADDA_DL_CH1 DL_24CH_CH1	feeds the codec DAC from the speaker's stream	set to 1 while playing
RCV Mux	routes the DAC to the earpiece	Voice Playback while playing
aw_dev_0_switch	speaker amp on/off (HAL turns it off for headphones and Bluetooth)	the guard; solo turns it off
aw_dev_0_volume	speaker amp attenuation, 1/8 dB steps (0–720 = 0–90 dB)	held at the Stereo attenuation
dsp_*_runtime_en	audio DSP task switches	playback detection
ADDA_DL_GAIN	digital gain before the DAC (HAL: 63311/65535, −0.3 dB)	left alone (shared with headphones)

> TECHNICAL
>
> Amp slot: register 0x06 through the driver's reg attribute, found under /sys/bus/i2c/devices/* by its dbg_prof and reg nodes; written as 0x06 0x<value>. One Awinic amp, at I2C 6-0034.

12.4 System properties used

Property	Set by	Meaning
vendor.audiohal.cmf_rotation	patched HAL	Android's last rotation parameter
vendor.audiohal.cmf_swap	daemon	output flags of the outputs the HAL swaps at rotation 270
vendor.audiohal.cmf_spk_ceil	daemon	speaker peak ceiling, 1/100 dB of full scale (0 = off)
vendor.audiohal.cmf_bass_ceil	daemon	bass guard ceiling (0 = off)
vendor.audiohal.cmf_ear_ceil	daemon	earpiece limiter ceiling (−250 on, 0 off)
vendor.audiohal.cmf_ear_level	daemon	leveller strength 0–3
vendor.audiohal.cmf_prot	library	1 once the library has processed its first speaker buffer
ctl.restart vendor.audio-hal	live switch	restarts the audio HAL
init.svc.vendor.audio-hal, init.svc_debug_pid.vendor.audio-hal	Android	HAL state and process id, read by the live switch and the crash-loop guard

12.5 The HAL patch

Four places in a stock audio.primary.mt6878.so / audio.primary.mediatek.so are patched:

Offset	Function	Change
0x27b820	doStereoToMonoConversionIfNeed	no fold to mono; loads the protection library once and calls it for every buffer; without it, the rotation swap alone (hal-swap/patch.S, 432 bytes)
0x2587c0	setParameters, rotation key	branch to the rotation store, which keeps the value in vendor.audiohal.cmf_rotation
0x27f2d4	InitArsiTaskConfig	b.ne → nop (always set hw_info 0x4)
0x1e0a00	SetArsiTaskConfig	the same, in the DSP stream manager

The stock bytes of the first two places are written first, so any earlier Stereo HAL patches to the same file, and the reverse gives back the stock file from any of them (verified to the stock hash). The patch is assembled and tested in an ARM64 emulator (hal-swap/run.sh).

12.6 Files Stereo replaces, and their hashes

File	Stock SHA-256	Stereo SHA-256
/vendor/lib64/hw/audio.primary.mt6878.so, audio.primary.mediatek.so	28db2e24e9d4c4f88a67a7631ab581fa064bb08a47a87314b7599a2cb757a324	a6362995ca0a3987e58e149c07057 3b4fd8ae4aecb66c196f9fdbfbf97a803ea

/vendor/etc/aurisys_config_rv.xml	f5818d29d40ce6a5e18765af96ab72a00db8c816e5a0c308c8003eff6e65f2cc	f0f7512085d86e8707131a4e6b3f413d25115c88419c69193493957da4d30ced
libcmfstereo.so (module's own)	—	bddc81fe27aa92df289446299dc30c6b2356b14fa0b9ed9178041924629f64cc

Stereo HAL files from v0.9.0–v1.0.3 and the pre-release test module's DSP configuration are also recognised and converted. The patched aurisys_config_rv.xml carries a marker comment, cmf_stereo: stereo, which is how the module tells that Stereo is what the system is running.

12.7 tinymix

Mixer controls cannot be read or written without a tinymix binary, and the CMF Phone 1 ships none. The module bundles one: a statically linked aarch64 build of upstream tinyalsa, hardware backend only, no dlopen.

```
upstream: https://github.com/tinyalsa/tinyalsa
commit:   9fab97ca07184371ecad81154d1dadb09d0fa7cf
license:  BSD-3-Clause

git clone https://github.com/tinyalsa/tinyalsa && cd tinyalsa
git checkout 9fab97ca07184371ecad81154d1dadb09d0fa7cf
aarch64-linux-gnu-gcc -static -O2 -Wall -Iinclude \
    src/mixer.c src/mixer_hw.c src/limits.c utils/tinymix.c -o tinymix
aarch64-linux-gnu-strip tinymix
```

If you would rather not trust a shipped binary, build it yourself and drop it at `/data/adb/cmf-stereo/bin/tinymix` (`chmod 0755`). Termux's `tinyalsa` package works too.

> TECHNICAL
>
> Both the tinyalsa 1.x `tinymix NAME VALUE` and the 2.x `tinymix set NAME VALUE` calling styles are handled, detected through `--help` rather than guessed from an error message, once per process. Search order in the code: the module's `bin/`, `/data/adb/cmf-stereo/bin`, `/vendor/bin`, `/system/bin`, `/system/vendor/bin`, `/odm/bin`, `/data/local/tmp`, then `$PATH`. The project README gives a different order, with `/data/adb/cmf-stereo/bin` first and a vendor-supplied `tinymix` winning over the bundled one; in this version's code the module's own `bin/tinymix` is checked first, so a copy you place in `/data/adb/cmf-stereo/bin` is used only when the module's own is missing or not executable.

12.8 The probe's output files

`stereoctl probe` writes `/sdcard/cmf-stereo-probe-<timestamp>/` (and a `.tar.gz` of it). Nothing on the device is modified.

File	Contents
`00-device.txt`	build properties and audio-related properties
`01-alsa.txt`	sound cards, devices, PCMs, the `/proc/asound` tree, hw_params and status
`02-mixer.txt`	every mixer control (or `NO TINYMIX FOUND`.)
`03-candidates.txt`	controls whose names look like receiver, speaker, output stage or routing
`04-amplifier.txt`	smart-amp sysfs entries, I2C device names, loaded modules, codec kernel log

05-hal-configs/	copies of audio policy, device, mixer path, effects and platform files
06-speaker-port.txt	the speaker and earpiece devicePort blocks
07-hal.txt	HAL libraries, services, audio processes, sound effects, manifest entries
08-routing.txt	the head of dumpsys audio and dumpsys media.audio_flinger
09-audio-param/	MediaTek AudioParam XMLs, 2nd-loudspeaker hits, aaudio/mmap properties
10-safety.txt	receiver gain controls and their values, audio protection nodes
SUMMARY.txt	device, tinymix, counts of controls and hits, the file list

12.9 The module's files

Path	What
customize.sh	install: device check, config kept across updates, Stereo re-staged
post-fs-data.sh	before mounts: boot watchdog, unknown-build guard
service.sh	after boot: stock copies saved, the daemon started
action.sh	the Action button: Mono ⇔ Stereo
uninstall.sh	stop, restore, give the amp its gain back
scripts/common.sh	config, logging, tinymix access, playback detection, the routing actions, the card's text
scripts/channels.sh	Stereo: hashes and patches, staging, live switching, the amp, protection settings, crash-loop guard

scripts/stereo-daemon.sh	keeps routing, amp slot, attenuation, rotation swap and protection applied while playing
scripts/stereoctl	the command-line tool the apps and you call
scripts/probe.sh	the hardware dump
scripts/xml_patch.sh	the audio-policy overlay (opt-in, not for this device)
system/bin/stereoctl	the shim that puts stereoctl on the path
protect/libcmfstereo.so	the protection library
config/	default settings and the verified CMF Phone 1 routing
bin/tinymix	static aarch64 tinyalsa build
system/vendor/...	the Stereo overlay, present only when staged

The module lives in /data/adb/modules/cmf_stereo/.

12.10 The data folder's files

Everything the module keeps in /data/adb/cmf-stereo/:

File	Purpose
stereo.conf, actions.conf	your settings and routing

stereo.log, stereo.log.1	the log and its previous rotation
state/	saved original values; also amp_ours (the attenuation the module set) and hal_pids (crash-loop guard)
stock/	hash-checked copies of the stock audio files, for switching to Mono live
build_state	known, updated or absent
disabled	the kill switch (stereoctl off)
guard_bypass, solo_prev	guard bypassed; the speaker switch's value before solo
boot_pending	boot watchdog strikes
daemon.pid, daemon.lock/	the running daemon
snap.before, snap.after	the two snapshots of stereoctl diff
bin/	your own tinymix, if you add one

12.11 MediaTek's second-loudspeaker machinery

MediaTek's "2nd loudspeaker" machinery is present on this build, in /vendor/etc/audio_param/:

```
2nd Loudspeaker Compensation Filter (2nd-ACF)
2nd Loudspeaker High/Low Pass Filter Order
bes_loudness_Sep_LR_Filter
bes_loudness_L_* / bes_loudness_R_*  (separate L and R filter sets)
```

An earlier revision of the README claimed it was absent; that came from broken search patterns that reported a clean zero for a pattern that never matched. On the CMF Phone 1 every `bes_loudness_R_*` value and `bes_loudness_Sep_LR_Filter` read `0x0`: MediaTek's stock template, untuned. 2nd-ACF is a filter feature: it applies a different EQ to a second driver's channel; it does not create a route to one. Whether it is wired to a second driver on this hardware is unresolved. The module's Stereo does not use it.

12.12 Development checks

```
sh tests/run-tests.sh     # 198 checks; run under sh, dash, mksh and busybox sh
sh hal-swap/run.sh        # the HAL patch in qemu-aarch64 (needs clang, ld.lld, qemu)
sh protect/build.sh       # the library rebuilds byte for byte (needs the NDK)
./build.sh              # the flashable zip
```

Every vendor file is patched only after its hash proves it is the analysed build, and the result is hashed again; `hal-swap/run.sh` checks the bytes `channels.sh` writes against the assembled patch.

12.13 Test status of this version

Verified on the phone: the earpiece leveller (v1.0.5); Stereo with a left-only file, Dirac on and off; Mono; live switching from `stereoctl channels`, both apps and the Action button; Magisk's Update button; left and right following the rotation in the apps of section 3.10; "While playing" for games and music players; every protection setting; the earpiece at +8 dB; hi-res playback in Stereo.

Not tested on a phone yet:

- the apps following changes made outside them (v1.0.6), tested with a stand-in;
- boost 6 over a longer listen;
- the protection after a live switch from a Mono boot (by design it starts after a reboot);
- the safety nets that need a failure: crash-loop guard, boot watchdog, a failed live switch ending on the stock files (covered by tests on fake files);
- the next Nothing OS update: the unknown-build guard and re-staging during a module update;
- how the speaker holds up to long, loud Stereo listening.

Pending: adding the next Nothing OS build after checking its file hashes; a short clip of the two drivers splitting a game's or film's audio.

12.14 Version history

Version	Date	Change
v1.0.9-beta	8 Nov 2026	marked beta; module unchanged since v1.0.8 (version code 1090)
v1.0.8	7 Nov 2026	every text says left and right follow how you hold the phone
v1.0.7	6 Nov	`stereoctl off` no longer asks for a reboot in Mono; exact tested build in the README
v1.0.6	5 Nov	Nothing Settings and the tile follow every change at once
v1.0.5	5 Nov	optional earpiece leveller
v1.0.4	5 Nov	the module's own Stereo protection; crash-loop guard
v1.0.3	5 Nov	both speakers in games and every other app
v1.0.2	5 Nov	apps with their own audio effect (VLC) follow the screen
v1.0.1	5 Nov	left and right right again (v1.0.0 played them reversed)
v1.0.0	5 Nov	one swap for every app; played reversed, replaced
v0.9.9	5 Nov	fixed Stereo from the Action button crashing the audio service; reversed channels in some apps
v0.9.8	5 Nov	Action button, live status on the card, stock sound after a Nothing OS update
v0.9.7	5 Nov	listed as by Dr.Essence; first release offered as an update

v0.9.6	4 Nov	updates through the root manager; SHA-256 checksums
v0.9.5	4 Nov	left and right follow which way the phone is turned
v0.9.4	4 Nov	Mono and Stereo switch without a reboot; quieter kernel log
v0.9.3	4 Nov	instant on/off; lighter in the background; `status brief`
v0.9.2	3 Nov	first true stereo; earpiece gain fixed (31 was mute)
v0.8.2, v0.7.1	Sep 2026	the earlier version: the same mono mix on both drivers

The full history of findings, tests and fixes is in `CHANGELOG.md`.

13

CREDITS AND LICENCE

Who made this, whose work it started from, and the terms you may use it under.

13.1 Author

Dr.Essence: author and maintainer. Ideas, design decisions, and every test on a real phone (logs, recordings, measurements) that the work rests on.

Code written with Claude Code, against a real CMF Phone 1: every routing claim came from measurements on the device, not from documentation.

13.2 The Hi-Res Audio module

This module owes a real debt to **Hi-Res Audio** by Adinata (@Adivenxnataly). It is the reason the project knew where to look. Before it, "use the earpiece as a second speaker on a MediaTek phone" was a vague idea with no entry point. That module pointed at MediaTek's `2nd Loudspeaker` / `bes_loudness` machinery and at the earpiece `<devicePort>` channel mask; working out why those levers do not pan out on the CMF Phone 1 led to the routing that does.

It aims at a different goal — high-resolution output, not dual speakers — so it is worth trying on its own terms. What this module took from it: edit the element you mean to change, validate the result, and keep a way back. Hence the block-aware rewrite, the XML structural check, and the boot watchdog.

> WARNING
>
> Do not run both modules at the same time: they touch the same files.

13.3 A bug reported upstream

Tested against Hi-Res Audio v2.0 with aaudio_mmap, versionCode 20000, commit 78706f5 (2025-06-12), it bootlooped a CMF Phone 1. The cause is a one-character typo in its customize.sh:

```
name="Apply Same Filter Setting with 2nd Loudspeaker\/>
```

The attribute value is never closed. The fix:

```
name="Apply Same Filter Setting with 2nd Loudspeaker\"\/>
```

Without it, Playback_ParamTreeView.xml gets <Field … name="Apply Same Filter Setting with 2nd Loudspeaker/>; MediaTek's AudioParamParser cannot parse that, so the audio HAL aborts on every boot. The sed fires on any device carrying a bes_loudness_L_lpf_order field.

Also worth knowing on a phone like this: it forces aaudio.mmap_policy=3 / aaudio.mmap_exclusive_policy=3 plus killall audioserver, which needs a HAL with an MMAP PCM path, and it sets the speaker channel mask to AUDIO_CHANNEL_OUT_SURROUND, which not every speaker port accepts. An earlier claim in this project's README that its <mixPort> insertions land mid-element was wrong and is retracted: it computes the correct insertion point and otherwise skips.

13.4 Why its stereo approach does not work here

Not a criticism: the approach is sound on hardware with a second loudspeaker. Two things stop it on the CMF Phone 1:

- `Playback_ParamTreeView.xml` is the view definition for MediaTek's tuning tool. The runtime coefficients live in `PlaybackACF_AudioParam.xml`. Editing the view changes which fields a tuning tool displays, not what the DSP does.
- Every `bes_loudness_R_*` value and `bes_loudness_Sep_LR_Filter` read `0x0`, MediaTek's untuned template, because this phone has no second loudspeaker. And 2nd-ACF is a filter, not a route.

13.5 Others who did this before

Thanks to the people who worked out this mod on other hardware and wrote it up: the OnePlus 6 stereo speaker mod, Xiaomi Merlin (`merlin_DualSpeaker_mod`), LeEco Le Max 2 (`Dual-Speaker-x2`) and whyred (Magisk dual speaker mod). Their write-ups of duplicated-mono "dual speaker" mods are what this project set out to go beyond: Mono mode is that; Stereo mode is real left and right.

13.6 Licence

The module is © 2026 Dr.Essence under CC BY-NC-SA 4.0. You may use, copy, change and share it, as long as you:

- credit Dr.Essence;
- do not use it commercially, including selling it or anything built from it;
- share changed versions under the same licence.

Releases up to v0.9.6 were published under OBSD; copies obtained under it keep those terms.

bin/tinymix is a statically linked aarch64 build of tinyalsa, under BSD-3-Clause. Its licence, upstream commit and exact build command are in bin/README.md, so you can reproduce it rather than trust the shipped binary.

©DR.ESSENCE · NDOT FONTS ARE OWNED BY ©NOTHING

CMF TRUE STEREO · V1.0.9-BETA · FREE AND OPEN SOURCE